Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Zarządzanie w systemach i sieciach komputerowych - projekt

Mechanizmy zarządzania wątkami programowymi w zadaniu wykrywania krawędzi na obrazach cyfrowych z wykorzystaniem operatorów gradientowych

Autorzy:

Karol Gąsior
Indeks: 280913

Mateusz Kunicki
Indeks: 272468

Prowadzący zajęcia:

Dr inż. Robert Wójcik, W4N

Ocena pracy:

Wrocław, 2026

Spis treści

1 Temat i cel projektu **3**

2 Sformułowanie problemu **3**

3 Analiza złożoności obliczeniowej problemu **4**

4 Metoda i algorytmy rozwiązywania problemu **6**
4.1 Wersja W1 – sekwencyjna (punkt odniesienia) 6
4.2 Wersja W2 – optymalizacja algorytmu i dostępu do pamięci (1 rdzeń) . . . 6
4.3 Wersja W3 – wektoryzacja SIMD (1 rdzeń) 7
4.4 Wersja W4 – wątki na wielu rdzeniach procesora 7
4.5 Wersja W5 – karta graficzna (OpenCL) 8
4.6 Zestawienie wersji 9

5 Metoda, technologie i narzędzia implementacji **9**

6 Sposób testowania i oceny jakości rozwiązań **10**

7 Realizacja projektu i wyniki **11**
7.1 Implementacja 11
7.2 Metodyka pomiarów 12
7.3 Poprawność 12
7.4 Porównanie wersji – sam algorytm 12
7.5 Rozkład czasu: wczytanie, algorytm, zapis 14
7.6 Wersja W2 i W3 – rozmiar maski 17
7.7 Wersja W4 – zarządzanie wątkami 18
7.7.1 Liczba wątków 18
7.7.2 Strategia przydziału i rozmiar fragmentu 19
7.7.3 Synchronizacja przy wyznaczaniu maksimum 20
7.7.4 Bariera wynikająca ze skalowania 21
7.7.5 Pula wątków a tworzenie wątków 21
7.7.6 Przypisanie wątków do rdzeni 22
7.7.7 Cały program w funkcji liczby wątków 23
7.8 Wersja W5 – karta graficzna 23
7.9 Rozmiar danych wejściowych 24
7.10 Jakość wykrywania krawędzi a rozmiar maski 27
7.11 Propozycje dalszych przyspieszeń (AI) 29
7.12 Podsumowanie i wnioski 30

1 Temat i cel projektu

Temat: Mechanizmy zarządzania wątkami programowymi w zadaniu wykrywania krawędzi na obrazach cyfrowych z wykorzystaniem operatorów gradientowych (Sobela i pokrewnych).

Celem projektu było zaprojektowanie i zaimplementowanie programu wykrywającego krawędzie na obrazie cyfrowym w pięciu wersjach oraz zmierzenie, ile czasu przetwarzania oszczędza każdy rodzaj przyspieszenia. Wersja W1 jest prostą implementacją sekwencyjną i stanowi punkt odniesienia. Każda z wersji W2–W5 bierze kod W1 i wprowadza do niego jeden rodzaj przyspieszenia. Wersja W2 nie korzysta z równoległości: zmienia algorytm i sposób korzystania z pamięci, aby sprawdzić, ile daje sama optymalizacja kodu na jednym rdzeniu. Wersje W3–W5 odpowiadają kolejnym poziomom przetwarzania równoległego [3]: równoległości na poziomie instrukcji procesora, czyli SIMD (W3), równoległości wątków na wielu rdzeniach procesora (W4) i masowej równoległości danych na setkach rdzeni karty graficznej (W5). Wersji nie łączono ze sobą. Każdą porównano z W1, więc zmierzony zysk pokazuje wkład jednego mechanizmu, a nie sumę kilku.

Zakres projektu objął:

- maski Sobela 3×3, 5×5 i 7×7, Prewitta, Scharra i Robertsa;
- obrazy od 0,04 do 59 Mpix (zdjęcia i obrazy syntetyczne);
- w wersji wielowątkowej W4 – mechanizmy zarządzania wątkami: liczbę wątków od 1 do 24 (oraz 32 i 48), cztery strategie przydziału pracy, rozmiar przydzielanego fragmentu, sposób synchronizacji wątków przy wyznaczaniu wartości maksymalnej, pulę wątków w porównaniu z tworzeniem wątków przy każdym etapie i przypisanie wątków do rdzeni procesora;
- w wersji GPU W5 – liczbę wątków uruchamianych na karcie graficznej, rozmiar grupy roboczej i koszt przesyłania danych między pamięcią komputera a kartą;
- dwa pomiary każdej wersji: samego algorytmu oraz całego programu z wczytaniem i zapisem obrazu;
- ocenę, czy większa maska daje lepszy wynik wykrywania krawędzi, na obrazach o znanym położeniu krawędzi z dodanym szumem;
- propozycje dalszych przyspieszeń przygotowane z pomocą modelu sztucznej inteligencji.

2 Sformułowanie problemu

Problem rozwiązywano przy następujących założeniach:

- obraz wejściowy jest wczytywany z pliku (JPG, PNG, GIF) i konwertowany do skali szarości o 8 bitach na piksel albo generowany syntetycznie w pamięci;
- cały obraz mieści się w pamięci operacyjnej;
- piksele leżące poza brzegiem obrazu zastępuje się najbliższym pikselem brzegowym (powielenie krawędzi);
- wynikiem jest obraz 8-bitowy o tych samych wymiarach co wejście;

- najmniejszą jednostką podziału pracy pomiędzy wątki procesora jest wiersz obrazu, a pomiędzy wątki karty graficznej – piksel;
- wszystkie wersje muszą dawać wynik identyczny co do bitu z wynikiem wersji W1.

Problem obliczeniowy [1, 2]: dla obrazu I o wymiarach $W \times H$ i pary masek K_x, K_y o rozmiarze $k \times k$ wyznaczyć dla każdego piksela moduł gradientu

$$G(x,y) = \sqrt{G_x(x,y)^2 + G_y(x,y)^2}, \qquad G_x = K_x * I, \qquad G_y = K_y * I, \tag{1}$$

gdzie $*$ oznacza dwuwymiarowy splot dyskretny

$$(K * I)(x,y) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K(i,j)\; I(x + j - \lfloor k/2 \rfloor,\; y + i - \lfloor k/2 \rfloor)\,, \tag{2}$$

a następnie przeskalować wynik do zakresu 0–255: $O(x,y) = \operatorname{round}\left(255 \cdot G(x,y) / \max G\right)$. Skalowanie względem największej wartości w obrazie wymaga znajomości $\max G$, czyli zakończenia obliczeń dla całego obrazu przed zapisaniem pierwszego piksela wyniku. Jako warianty pomocnicze zbadano skalowanie stałe (względem teoretycznego maksimum maski) i obcięcie wartości większych od 255, które tego wymagania nie mają.

Problem decyzyjny: czy dla danego obrazu, maski i sprzętu istnieje wersja programu i jej konfiguracja (liczba wątków, strategia przydziału pracy, rozmiar fragmentu, rozmiar grupy roboczej GPU), dla której czas przetwarzania nie przekracza zadanej wartości $T_{\max}$?

Problem optymalizacyjny [3]: dla każdej wersji dobrać parametry tak, aby zapewnić optymalne wartości następujących kryteriów:

- czas zegarowy przetwarzania obrazu – minimum;
- przyspieszenie względem wersji W1, $S = T_{W1}/T_{Wi}$ – maksimum;
- dla wersji wielowątkowej: przyspieszenie $S(N) = T(1)/T(N)$ i efektywność $E(N) = S(N)/N$ – maksimum; nierównowaga obciążenia wątków (różnica czasów zakończenia pracy) – minimum;
- zgodność wyniku z wersją W1 – wynik identyczny co do bitu.

Osobnym kryterium jest jakość wykrywania krawędzi. Zależy ona od wyboru maski, a nie od wersji programu, ponieważ wszystkie wersje dają ten sam wynik. Dlatego zbadano ją oddzielnie, w punkcie 7.10.

3 Analiza złożoności obliczeniowej problemu

Wyznaczenie modułu gradientu jest problemem z klasy P: istnieje algorytm deterministyczny o złożoności wielomianowej, który oblicza wynik dokładnie, bez przeszukiwania przestrzeni rozwiązań. Splot dwuwymiarowy wymaga dla każdego piksela k^2 mnożeń z dodawaniem dla G_x i tyle samo dla G_y, czyli $2k^2$ operacji. Całe zadanie ma więc złożoność $O(WHk^2)$, liniową względem liczby pikseli. Maski Sobela, Prewitta i Scharra są rozdzielne: każdą z nich można zapisać jako iloczyn wektora wygładzającego s i różnicującego d, np. dla Sobela 3×3:

$$K_x = s^T d = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}, \qquad K_y = d^T s. \tag{3}$$

Splot z maską rozdzielną można wtedy policzyć jako dwa sploty jednowymiarowe: poziomy (dla każdego wiersza obrazu wektorami s i d) i pionowy (na wynikach przebiegu poziomego). Wymaga to $4k$ zamiast $2k^2$ operacji na piksel, co daje złożoność $O(WHk)$: dla $k = 3$ jest to 12 zamiast 18 operacji, a dla $k = 7$ – 28 zamiast 98. Maska Robertsa nie jest rozdzielna. Skalowanie względem maksimum wymaga dodatkowego przebiegu $O(WH)$ i redukcji maksimum. Złożoność pamięciowa wynosi $O(WH)$: obraz wejściowy (1 B/piksel), bufor modułu gradientu (4 B/piksel) i wynik (1 B/piksel), łącznie około 6 B/piksel.

Wartość każdego piksela wyniku zależy wyłącznie od jego otoczenia w obrazie wejściowym, więc zadanie jest łatwo zrównoleglane i teoretycznie pozwala na przyspieszenie liniowe względem liczby jednostek obliczeniowych. Przyspieszenie ogranicza jednak część sekwencyjna programu, zgodnie z prawem Amdahla [6, 3]:

$$S(N) \le \frac{1}{s + \dfrac{1-s}{N}}, \qquad \lim_{N\to\infty} S(N) = \frac{1}{s}, \tag{4}$$

gdzie s oznacza udział części sekwencyjnej w czasie wykonania na jednej jednostce. W tym zadaniu część sekwencyjną tworzą: wczytanie i dekodowanie pliku obrazu, zapis wyniku, przydział buforów, redukcja maksimum i bariera między etapami, a w wersji GPU – przesłanie obrazu na kartę i wyniku z powrotem. Drugim ograniczeniem jest przepustowość pamięci. Splot 3×3 wykonuje około 18 operacji na piksel, a jednocześnie przesyła z pamięci i do pamięci około 6 B na piksel, czyli zaledwie około 3 operacje na bajt danych. Po przyspieszeniu obliczeń czas może więc zależeć bardziej od szybkości odczytu i zapisu pamięci niż od mocy obliczeniowej, szczególnie gdy obraz nie mieści się w pamięci podręcznej.

Liczbę operacji i sposób zrównoleglenia w poszczególnych wersjach zestawiono w tabeli 1.

Tabela 1: Złożoność i równoległość wersji programu (obraz $W \times H$, maska $k \times k$, N wątków procesora, P rdzeni karty graficznej).

Wersja	Operacje na piksel	Równoległość	Czas (rząd)
W1	$2k^2$ mnożeń, $2k^2$ obcięć współrzędnych	brak	$O(WHk^2)$
W2	$4k$ mnożeń (rozdzielne), bez obcięć we wnętrzu	brak	$O(WHk)$
W3	$2k^2$ mnożeń wykonywanych po 8 naraz	8 pikseli na instrukcję (AVX2)	$O(WHk^2/8)$
W4	$2k^2$ mnożeń, jak W1	N wątków na rdzeniach CPU	$O(WHk^2/N)$ + część sekwencyjna
W5	$2k^2$ mnożeń, jak W1	do kilku milionów wątków GPU na 768 rdzeniach	$O(WHk^2/P)$ + przesył danych

4 Metoda i algorytmy rozwiązywania problemu

4.1 Wersja W1 – sekwencyjna (punkt odniesienia)

Wersja W1 realizuje wzór (1) wprost, w jednym wątku. Dla każdego piksela (x, y) obrazu program:

1. przechodzi po wszystkich $k \times k$ polach maski; dla każdego pola wyznacza współrzędne piksela źródłowego i obcina je do zakresu obrazu funkcją `std::clamp` (także wtedy, gdy piksel leży daleko od brzegu i obcięcie nie jest potrzebne);
2. odczytuje współczynniki K_x i K_y z tablicy dynamicznej (`std::vector`), której rozmiar jest znany dopiero w czasie wykonania, i sumuje iloczyny w zmiennych całkowitych G_x i G_y;
3. oblicza moduł $\sqrt{G_x^2 + G_y^2}$ w arytmetyce podwójnej precyzji i zapisuje go jako liczbę `float` do bufora modułu, aktualizując przy tym bieżące maksimum.

Po przetworzeniu całego obrazu drugi przebieg mnoży każdą wartość bufora przez $255/\max G$, zaokrągla ją i zapisuje jako bajt wyniku. Bufor modułu i bufor wyniku są przydzielane i zerowane przy każdym wywołaniu. Wersja ta jest najprostszą poprawną implementacją, a jej wynik jest wzorcem, z którym porównano wszystkie pozostałe wersje.

Moduł liczony jest w podwójnej precyzji celowo: iloczyny G_x^2 i G_y^2 są wtedy dokładne, a pierwiastek poprawnie zaokrąglony, więc wynik nie zależy od kolejności działań ani od tego, czy kompilator połączy mnożenie z dodawaniem w jedną instrukcję. Pozwoliło to wymagać od każdej wersji wyniku identycznego co do bitu.

4.2 Wersja W2 – optymalizacja algorytmu i dostępu do pamięci (1 rdzeń)

Wersja W2 działa na jednym rdzeniu i nie używa instrukcji wektorowych w sposób jawny. Wprowadza cztery zmiany w samym algorytmie i organizacji danych:

1. **Obsługa brzegu poza główną pętlą.** Obcinanie współrzędnych jest potrzebne tylko w pasach szerokości $\lfloor k/2 \rfloor$ przy brzegach obrazu. Wnętrze obrazu jest przetwarzane osobną pętlą bez `std::clamp`, a wskaźniki do k wierszy źródłowych są wyznaczane raz na wiersz wyniku zamiast przy każdym odczycie. Znikają rozgałęzienia i przeliczanie indeksów w najczęściej wykonywanym fragmencie kodu.
2. **Maska jako stała czasu kompilacji.** Rozmiar maski i jej współczynniki są parametrami szablonu C++ (`constexpr`). Kompilator rozwija pętle po masce, pomija mnożenia przez zero (dla Sobela 3×3 trzy z dziewięciu współczynników każdej maski są zerowe) i zamienia mnożenia przez małe stałe na tańsze operacje.
3. **Splot rozdzielny.** Dla masek rozdzielnych każdy wiersz obrazu jest najpierw splatany poziomo z wektorami s i d, a wyniki k ostatnich wierszy są przechowywane w buforze pierścieniowym. Splot pionowy korzysta z tego bufora. Każdy wiersz źródłowy jest przetwarzany poziomo tylko raz, a liczba operacji spada z $2k^2$ do $4k$ na piksel. Bufor k wierszy mieści się w pamięci podręcznej L1/L2 procesora. Maska Robertsa, która nie jest rozdzielna, jest liczona splotem dwuwymiarowym z punktów 1 i 2.

4. **Mniejsze typy i bufory wielokrotnego użytku.** Wyniki przebiegu poziomego mieszczą się w 16 bitach (dla Sobela 7×7 co najwyżej $255 \cdot 64 = 16\,320$), więc bufor pierścieniowy przechowuje liczby `int16` zamiast `int32`, co zmniejsza o połowę ilość przesyłanych danych. Bufory są wyrównane do 64 B (rozmiar linii pamięci podręcznej) i przydzielane raz, a nie przy każdym wywołaniu, co usuwa koszt przydziału i pierwszego zapisu stron pamięci.

Ponieważ zysk ze splotu rozdzielnego rośnie z rozmiarem maski, przewidywano, że W2 będzie zyskiwać względem W1 tym więcej, im większe jest k.

4.3 Wersja W3 – wektoryzacja SIMD (1 rdzeń)

Wersja W3 zachowuje algorytm W1: splot dwuwymiarowy $k \times k$, współczynniki maski odczytywane z tablicy w czasie wykonania (bez pomijania zer i bez rozdzielności), ten sam sposób liczenia modułu, przydział buforów przy każdym wywołaniu. Zmianą jest wykonywanie obliczeń dla 8 pikseli jednocześnie instrukcjami wektorowymi AVX2 [8], czyli równoległość na poziomie instrukcji procesora:

- dla każdego pola maski 8 sąsiednich pikseli (8 bajtów) jest wczytywanych jedną instrukcją i rozszerzanych do ośmiu liczb 32-bitowych w 256-bitowym rejestrze (`_mm256_cvtepu8_epi32`);
- współczynnik maski jest powielany do wszystkich 8 pozycji rejestru, a mnożenie i dodawanie (`_mm256_mullo_epi32`, `_mm256_add_epi32`) wykonują się dla 8 pikseli w jednej instrukcji;
- moduł jest liczony wektorowo: pierwiastkiem z 8 liczb pojedynczej precyzji (`_mm256_sqrt_ps`), gdy $G_x^2 + G_y^2 < 2^{24}$ i wynik jest dokładny, a w przeciwnym razie 2 razy po 4 liczby podwójnej precyzji (`_mm256_sqrt_pd`);
- drugi przebieg (skalowanie do 0–255) również przetwarza 8 pikseli na instrukcję.

Instrukcji wektorowej nie da się zastosować do pikseli brzegowych, dla których każdy z 8 pikseli wymagałby innego obcięcia współrzędnych. Dlatego kolumny brzegowe są liczone kodem W1, a w ścieżce wektorowej obcinany jest tylko numer wiersza źródłowego (raz na wiersz maski). Ta zmiana nie jest celem W3, ale została wymuszona przez wektoryzację. Ma jednak skutek uboczny: ścieżka wektorowa nie obcina współrzędnych kolumn we wnętrzu obrazu, więc część zysku W3 pochodzi z usunięcia tych obcięć, czyli z optymalizacji, którą W2 wprowadza osobno. Teoretyczna granica przyspieszenia samej wektoryzacji wynosi 8; w praktyce zmniejsza ją koszt rozszerzania bajtów do 32 bitów, część skalarna przy brzegach i przepustowość pamięci.

4.4 Wersja W4 – wątki na wielu rdzeniach procesora

Wersja W4 wykonuje kod W1 bez zmian (to samo jądro obliczeniowe), ale dzieli obraz pomiędzy N wątków. Zastosowano dekompozycję danych: obraz jest dzielony na fragmenty złożone z kolejnych wierszy, a każdy wątek zapisuje wyniki do rozłącznego obszaru pamięci, więc same obliczenia nie wymagają blokad. Przetwarzanie przebiega w dwóch etapach rozdzielonych barierą:

1. splot i wyznaczenie lokalnego maksimum przez każdy wątek;
2. redukcja lokalnych maksimów do maksimum globalnego, a następnie równoległe skalowanie wyniku.

Wątki są tworzone raz i utrzymywane w puli (*thread pool*): między etapami czekają uśpione na zmiennej warunkowej (`std::condition_variable`) i są budzone, gdy pojawia się praca. Przydziałem fragmentów zarządza scheduler oferujący cztery strategie wzorowane na klauzuli `schedule` standardu OpenMP [5]:

- *static* – obraz dzielony na N równych bloków, po jednym na wątek; brak narzutu, ale wątek opóźniony przez system wydłuża cały etap;
- *cyclic* – fragmenty po c wierszy przydzielane kolejnym wątkom po kolei (fragment i dla wątku $i \bmod N$);
- *dynamic* – wolny wątek pobiera następny fragment ze wspólnego licznika atomowego; dobre równoważenie obciążenia kosztem operacji atomowej na każdy fragment;
- *guided* – jak *dynamic*, ale rozmiar fragmentu maleje wraz z ilością pozostałej pracy (duże fragmenty na początku, małe na końcu).

W wersji W4 zbadano następujące mechanizmy zarządzania wątkami:

- liczbę wątków N od 1 do 24 oraz 32 i 48 (więcej wątków niż rdzeni logicznych), z uwzględnieniem topologii procesora: 12 rdzeni fizycznych po 2 wątki sprzętowe (SMT), podzielonych na 2 bloki (CCD) po 6 rdzeni z osobną pamięcią podręczną L3;
- strategię przydziału i rozmiar fragmentu c od 1 do 256 wierszy;
- sposób wyznaczania maksimum globalnego: lokalne maksimum każdego wątku i redukcja po barierze; porównanie każdego piksela ze wspólną zmienną atomową (`std::atomic`, zapis operacją porównaj-i-zamień); aktualizacja wspólnej zmiennej pod blokadą `std::mutex` dla każdego piksela; zapis bieżącego maksimum do własnej komórki wątku przy każdym pikselu – z komórkami w osobnych liniach pamięci podręcznej albo w jednej linii, co wywołuje fałszywe współdzielenie (*false sharing*) [7];
- pulę wątków w porównaniu z tworzeniem nowych wątków (`std::thread`) w każdym etapie;
- przypisanie wątków do rdzeni logicznych (`pthread_setaffinity_np`): bez przypisania (decyduje system), najpierw jeden blok rdzeni, naprzemiennie oba bloki, tylko jeden wątek na rdzeń fizyczny;
- koszt bariery wynikającej ze skalowania względem maksimum, w porównaniu ze skalowaniem stałym, które wymaga tylko jednego etapu.

4.5 Wersja W5 – karta graficzna (OpenCL)

Wersja W5 przenosi algorytm W1 bez zmian na kartę graficzną NVIDIA GeForce GTX 1050 Ti (768 rdzeni CUDA w 6 multiprocesorach) przy użyciu standardu OpenCL [9]. Każdy wątek karty (*work-item*) oblicza jeden piksel: czyta $k \times k$ pikseli z pamięci globalnej karty, mnoży je przez współczynniki maski z pamięci stałej i oblicza moduł. Przebieg programu:

1. przesłanie obrazu z pamięci komputera do pamięci karty przez magistralę PCIe;
2. kernel splotu; w każdej grupie roboczej wątki wyznaczają wspólne maksimum w pamięci lokalnej, a jeden wątek grupy aktualizuje maksimum globalne operacją atomową;

3. odczyt maksimum przez procesor i obliczenie skali;
4. kernel skalowania do 0–255;
5. przesłanie wyniku z powrotem do pamięci komputera.

Inicjalizacja (wybór urządzenia, kompilacja kernela) i przydział buforów na karcie odbywają się raz, przed pomiarem.

Liczba rdzeni karty jest stała, ale programista wybiera liczbę wątków uruchamianych na karcie oraz rozmiar grupy roboczej (wątków współdzielących pamięć lokalną i wykonywanych na jednym multiprocesorze). Wątek może liczyć kilka pikseli: piksele $x_0, x_0 + G, x_0 + 2G, \ldots$, gdzie G to liczba wątków w wierszu, dzięki czemu sąsiednie wątki zawsze czytają sąsiednie piksele i odczyty z pamięci łączą się w większe transakcje. Karta tej klasy liczy w podwójnej precyzji około 32 razy wolniej niż w pojedynczej, dlatego moduł jest liczony w podwójnej precyzji tylko wtedy, gdy pojedyncza nie zapewnia wyniku dokładnego; dodatkowo zmierzono wariant zawsze w pojedynczej precyzji.

4.6 Zestawienie wersji

Tabela 2 zestawia, co zmienia każda wersja względem W1 i jakie czynniki przewidywano jako ograniczenia jej przyspieszenia.

Tabela 2: Wersje programu: rodzaj przyspieszenia i przewidywane ograniczenia.

Wersja	Co zmienia względem W1	Poziom równoległości [3]	Przewidywane ograniczenia
W1	–	brak	–
W2	brzegi poza pętlą, maska w czasie kompilacji, splot rozdzielny, `int16`, bufory wielokrotnego użytku	brak (1 rdzeń)	niewielki zysk rozdzielności dla $k = 3$; drugi przebieg skalowania
W3	8 pikseli na instrukcję (AVX2), skalowanie wektorowe	poziom instrukcji (SIMD)	rozszerzanie bajtów, brzegi skalarne, pamięć
W4	podział wierszy pomiędzy N wątków, pula, scheduler	poziom zadań (wątki, rdzenie)	część sekwencyjna, bariera, SMT, przepustowość pamięci
W5	obliczenia na karcie graficznej, 1 wątek na piksel	masowa równoległość danych	przesył PCIe, podwójna precyzja

5 Metoda, technologie i narzędzia implementacji

Program napisano w języku C++ (standard C++17) z wykorzystaniem biblioteki STL, w szczególności mechanizmów wielowątkowości: `std::thread`, `std::mutex`, `std::condition_variable` i `std::atomic` [4, 12]. Wersja W3 używa funkcji wewnętrznych kompilatora dla rozszerzenia AVX2 (`immintrin.h`) [8], kompilowanych tylko dla funkcji tej wersji, aby pozostała część programu działała na każdym procesorze x86-64. Wersja W5 używa OpenCL 1.2 ze sterownikiem karty NVIDIA. Odczyt i zapis plików graficznych realizują jednoplikowe biblioteki `stb_image` i `stb_image_write` [14].

Logika obliczeń jest zamknięta w bibliotece `edge_core`, w której wszystkie wersje są wywoływane przez jedną funkcję, co gwarantuje identyczne granice pomiaru czasu. Pomiary wykonuje konsolowa aplikacja `edge_bench` oparta na module Qt Core [13], uruchamiana z pliku opisującego eksperyment (JSON) i zapisująca surowe wyniki do plików CSV wraz z opisem warunków pomiaru (procesor, kompilator, flagi, wersja kodu). Projekt budowano za pomocą CMake i kompilatora GCC 13 w trybie Release (optymalizacja `-O3`). Skrypty w języku Python (biblioteki NumPy, pandas, Matplotlib, Pillow) uruchamiały eksperymenty kolejno, przetwarzały wyniki i generowały wszystkie wykresy, tabele i liczby zawarte w tym sprawozdaniu; dokumentację przygotowano w systemie LaTeX.

Programy uruchamiano na komputerze z procesorem AMD Ryzen 9 9900X i kartą graficzną NVIDIA GeForce GTX 1050 Ti (4 GB) pod systemem Ubuntu 24.04. Procesor ma 12 rdzeni fizycznych i 24 wątki sprzętowe (SMT). Rdzenie są podzielone na 2 bloki (CCD) po 6 rdzeni, a każdy blok ma własną pamięć podręczną L3 o pojemności 32 MB. Każdy rdzeń ma 1 MB pamięci L2. Procesor obsługuje rozszerzenia AVX2 i AVX-512. Ta budowa jest istotna przy interpretacji wyników W4: wątki z różnych bloków nie współdzielą pamięci L3.

6 Sposób testowania i oceny jakości rozwiązań

Poprawność. Wynik każdej wersji porównywano z wynikiem W1 dla tego samego obrazu, maski i sposobu skalowania. Zestaw testowy obejmował przypadki brzegowe: obraz 1×1, jedną kolumnę, jeden wiersz, obraz mniejszy od maski 7×7, mniej wierszy niż wątków i wymiary nieparzyste, a dla W4 i W5 – różne liczby wątków, strategie przydziału, rozmiary fragmentu i grupy roboczej. Kolejne wywołania otrzymywały bufor wyniku z poprzedniego wywołania, więc test wykrywał także wersję, która nie nadpisuje każdego piksela.

Pomiary czasu. Dla każdej wersji wykonano dwa pomiary:

1. **sam algorytm** – od obrazu w pamięci do gotowego obrazu wynikowego; dla W5 łącznie z przesłaniem danych na kartę i z powrotem;
2. **cały program** – wczytanie i zdekodowanie pliku obrazu, algorytm, zapis wyniku do pliku JPG.

Analiza skupia się na samym algorytmie, ale dla każdej wersji pokazano rozkład czasu całego programu na wczytanie obrazu, algorytm i zapis wyniku, ponieważ operacje plikowe są sekwencyjne. Zgodnie z prawem Amdahla (4) mogą one ograniczać zysk całego programu bardziej niż sam algorytm. Czas algorytmu rozbito dodatkowo na etapy (splot, skalowanie, a dla W5 przesył do karty, obliczenia i przesył z karty).

Każdą konfigurację wykonywano najpierw 1–2 razy bez zapisu wyniku (rozgrzewka: pierwsze wywołanie przydziela pamięć i ładuje kod do pamięci podręcznej), a następnie 11–15 razy z zapisem czasu. Jako wartość przyjęto medianę, a jako rozrzut – zakres od minimum do maksimum, zgodnie z zaleceniami Hoeflera i Belli [11]. Mediana jest mniej wrażliwa niż średnia na pojedyncze pomiary wydłużone przez system operacyjny. Czas mierzono zegarem monotonicznym `std::chrono::steady_clock`. Eksperymenty uruchamiano kolejno, nigdy dwa naraz, a każdy z nich rozpoczynał się tylko wtedy, gdy inne procesy zajmowały mniej niż 5 % czasu procesora. Regulator częstotliwości procesora pracował w trybie performance, czyli bez obniżania częstotliwości przy małym obciążeniu.

Każde przyspieszenie podano razem z czasem bezwzględnym wersji bazowej, aby było wiadomo, od jakiej wartości jest liczone.

Dane testowe. Zdjęcie NASA Blue Marble (21 600 × 10 800 pikseli) [15] przeskalowane do 14 rozmiarów od 256 × 144 do 10 240 × 5 760 (0,04–59 Mpix), obrazy o tej samej liczbie pikseli i różnych proporcjach, zdjęcia z zestawu Kodak [16] (768 × 512), obraz Lena (512 × 512) oraz obrazy syntetyczne z losowymi figurami. Duże obrazy służyły do pomiarów czasu, a małe zdjęcia – do sprawdzenia poprawności i do rysunków wyniku filtru. Rozdzielczości HD, 4K i 8K oznaczają odpowiednio 1920 × 1080, 3840 × 2160 i 7680 × 4320.

Jakość wykrywania krawędzi. Na syntetycznym obrazie z figurami o stałej jasności, dla którego położenie krawędzi jest znane co do piksela, dodano szum gaussowski o odchyleniu standardowym σ od 0 do 40 poziomów szarości. Wynik każdej maski zamieniono na obraz binarny progowaniem (próg dobierany osobno dla każdej maski i poziomu szumu tak, aby dawał najlepszy wynik) i porównano z prawdziwymi krawędziami miarą Pratta [10]:

$$F = \frac{1}{\max(N_I, N_D)} \sum_{j=1}^{N_D} \frac{1}{1 + \alpha\, d_j^2}, \qquad \alpha = \tfrac{1}{9}, \tag{5}$$

gdzie N_I – liczba prawdziwych pikseli krawędzi, N_D – liczba wykrytych, d_j – odległość j-tego wykrytego piksela od najbliższej prawdziwej krawędzi. $F = 1$ oznacza wykrycie idealne; miara karze zarówno brakujące i fałszywe krawędzie, jak i krawędzie przesunięte lub pogrubione.

7 Realizacja projektu i wyniki

7.1 Implementacja

Kod podzielono na trzy części. Biblioteka `edge_core` (C++17, bez zależności od Qt) zawiera wszystkie wersje algorytmu, pulę wątków, scheduler, kod OpenCL, weryfikację poprawności i pomiar czasu. Wszystkie wersje wywołuje jedna funkcja `VariantRunner::run(wersja, obraz, maska, konfiguracja)`, więc granice pomiaru są wspólne: od obrazu w pamięci do gotowego obrazu 0–255. Aplikacja konsolowa `edge_bench` (Qt Core) wczytuje opis eksperymentu w formacie JSON, wykonuje wszystkie konfiguracje kolejno i zapisuje każdy pomiar do pliku CSV na bieżąco. Skrypty w Pythonie uruchamiają eksperymenty jeden po drugim, odmawiają pomiaru przy obciążonym procesorze i generują rysunki, tabele oraz plik z liczbami, które w tym sprawozdaniu są wstawiane bezpośrednio z wyników pomiarów.

Wersje W1–W5 zrealizowano zgodnie z opisem w punkcie 4. W1 jest pierwotnym programem z założeń. W2 jest osobnym jądrem obliczeniowym z maską jako parametrem szablonu i splotem rozdzielnym na buforze pierścieniowym liczb `int16`. W3 jest nowym jądrem z funkcjami wewnętrznymi AVX2. W4 to pula wątków i scheduler wywołujące bez zmian jądro W1 dla przydzielonych bloków wierszy. W5 to kernel OpenCL, w którym liczbę pikseli na wątek i rozmiar grupy roboczej podaje się jako parametry uruchomienia.

Względem założeń wprowadzono dwie zmiany. Pierwsza dotyczy W3: ścieżka wektorowa nie obcina współrzędnych kolumn we wnętrzu obrazu (punkt 4.3), więc zmierzony zysk W3 obejmuje oprócz wektoryzacji także usunięcie tych obcięć. Druga dotyczy W5:

inicjalizacja OpenCL i przydział buforów na karcie odbywają się raz, przed pomiarem, a pierwsze, dłuższe wywołanie jest odrzucane jako rozgrzewka. W kodzie pozostały również wersje pośrednie z wcześniejszego etapu projektu (osobno każda optymalizacja W2, wersja z automatyczną wektoryzacją, wersja OpenMP, dwa dodatkowe kernele GPU). Nie są one opisywane w sprawozdaniu, ponieważ łączą kilka rodzajów przyspieszenia naraz i nie pozwalają ocenić wkładu jednego mechanizmu.

7.2 Metodyka pomiarów

Pomiary wykonano 9 listopada 2026 r. na sprzęcie opisanym w punkcie 5, pod systemem Ubuntu 24.04.5 LTS z jądrem Linux 7.0.0-42-generic. Program skompilowano kompilatorem gcc 13.3.0 z flagami `-Wall -Wextra -O3 -DNDEBUG`. W trakcie pomiarów nie uruchamiano innych programów. Cały przebieg, czyli kompilacja, weryfikacja poprawności, 15 eksperymentów i wygenerowanie rysunków, trwał około 17 minut. W porównaniach wersji W4 pracowała z 24 wątkami, harmonogramem *dynamic* i fragmentem 16 wierszy. Tę konfigurację wybrano na podstawie wcześniejszych pomiarów próbnych, a jej wpływ zbadano osobno w punkcie 7.7.

Liczba powtórzeń wynosiła 15 w większości eksperymentów i 11 w eksperymentach z wczytaniem i zapisem plików oraz z kartą graficzną, które trwają dłużej. Wszystkie czasy w tekście, na rysunkach i w tabelach są medianami. Słupki błędu na rysunkach obejmują zakres od minimum do maksimum.

Część konfiguracji miała współczynnik zmienności czasu powyżej 5 %, w najgorszym przypadku około 22 %. Dotyczyło to prawie wyłącznie wersji W4 przy czasach rzędu pojedynczych milisekund oraz bardzo małych obrazów: krótkie przerwy wynikające z pracy planisty systemu operacyjnego stanowią wtedy dużą część mierzonego czasu, a jeden opóźniony wątek opóźnia cały etap. Z tego powodu podawana jest mediana, a nie średnia, i dlatego różnic mniejszych niż około 10 % między konfiguracjami W4 nie interpretowano jako istotnych.

7.3 Poprawność

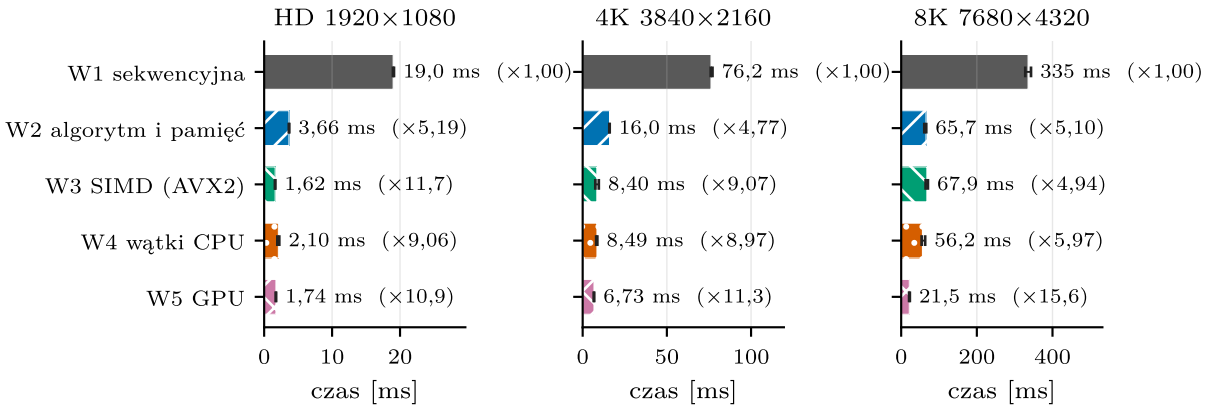
Przed pomiarami uruchomiono weryfikację wszystkich wersji względem W1. Obejmowała ona 10 368 przypadków: obrazy od 1×1 do zdjęć, 6 masek i 3 sposoby skalowania, a dla W4 i W5 także różne liczby wątków, harmonogramy, rozmiary fragmentu, rozmiary grupy roboczej i liczby pikseli na wątek GPU. Wszystkie wersje dały wynik identyczny co do bitu z W1. Dla W5 dopuszczono różnicę o 1 poziom szarości, ale w trybie dokładnym nie wystąpiła ani razu. W trybie zawsze pojedynczej precyzji największa różnica wyniosła 1 poziom szarości.

7.4 Porównanie wersji – sam algorytm

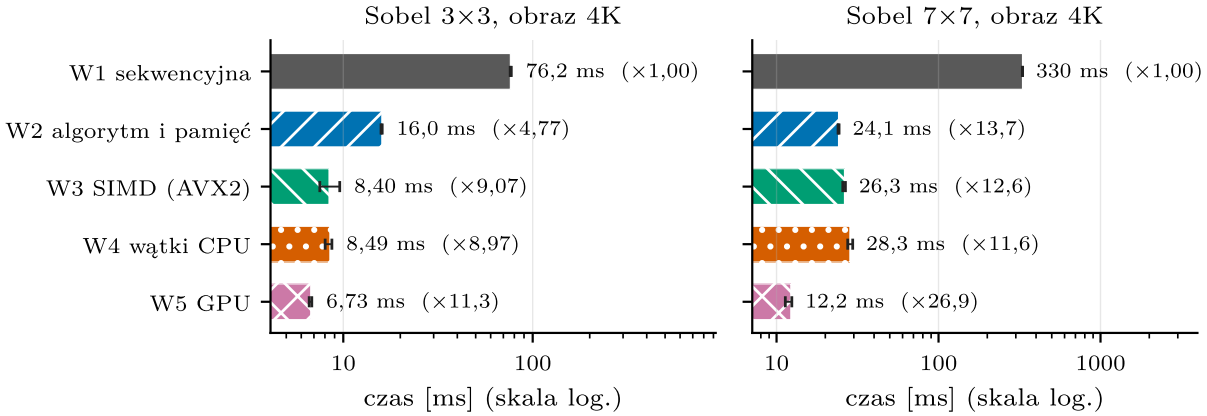
Czasy samego algorytmu dla wszystkich wersji przedstawiono na rysunku 1, a dla dwóch rozmiarów maski – na rysunku 2 i w tabeli 3.

Dla obrazu 4K i maski 3×3 wersja W1 potrzebowała 76,2 ms. Każda z pozostałych wersji była szybsza, ale żadna nie zbliżyła się do teoretycznej granicy swojego mechanizmu. W2 skróciła czas do 16,0 ms (4,77×), W3 do 8,40 ms (9,07×), W4 na 24 wątkach do 8,49 ms (8,97×), a W5 do 6,73 ms (11,3×).

Zysk W2 dla maski 3×3 nie pochodzi z rozdzielności (12 zamiast 18 operacji, czyli najwyżej 1,5×), ale z usunięcia obcięć współrzędnych i rozgałęzień z gorącej pętli oraz z



Rysunek 1: Czas samego algorytmu dla wersji W1–W5 (maska Sobela 3×3, obrazy HD, 4K i 8K); mediana z 15 powtórzeń, słupki błędu: min–maks; w nawiasie przyspieszenie względem W1. W4: 24 wątki.



Rysunek 2: Czas samego algorytmu dla masek Sobela 3×3 i 7×7 (obraz 4K); skala logarytmiczna.

Tabela 3: Czas samego algorytmu [ms] i przyspieszenie względem W1 (w nawiasie).

Wersja	HD, 3×3	4K, 3×3	8K, 3×3	4K, 7×7
W1 sekwencyjna	19,0 (1,00×)	76,2 (1,00×)	335 (1,00×)	330 (1,00×)
W2 algorytm i pamięć	3,66 (5,19×)	16,0 (4,77×)	65,7 (5,10×)	24,1 (13,7×)
W3 SIMD (AVX2)	1,62 (11,7×)	8,40 (9,07×)	67,9 (4,94×)	26,3 (12,6×)
W4 wątki CPU	2,10 (9,06×)	8,49 (8,97×)	56,2 (5,97×)	28,3 (11,6×)
W5 GPU	1,74 (10,9×)	6,73 (11,3×)	21,5 (15,6×)	12,2 (26,9×)

rozwinięcia pętli po stałej masce. Przyspieszenie W2 jest prawie takie samo dla obrazów HD, 4K i 8K (od 4,77× do 5,19×), ponieważ W2 przetwarza obraz wierszami i jej bufor roboczy (k wierszy) mieści się w pamięci podręcznej niezależnie od rozmiaru obrazu.

W3 wykonuje 8 mnożeń w jednej instrukcji, a dla obrazu HD osiągnęła 11,7×, czyli więcej niż wynosi granica samej wektoryzacji. Nadwyżka pochodzi z braku obcięć kolumn we wnętrzu obrazu (punkt 7.1). Dla obrazu 8K przyspieszenie spadło do 4,94×. Obraz 8K wraz z buforem modułu zajmuje około 200 MB i nie mieści się w pamięci L3 (2×32 MB), więc wektorowe jądro czeka na dane z pamięci operacyjnej. Zjawisko to opisano dokładniej w punkcie 7.9.

W4 na 24 wątkach przyspieszyła tylko 8,97× dla 4K i 5,97× dla 8K. Przyczyną jest część sekwencyjna: W4 wywołuje jądro W1, które przy każdym wywołaniu przydziela i zeruje bufor modułu (4 B/piksel) i bufor wyniku. Robi to jeden wątek przed rozdzieleniem pracy. Dla obrazu 8K czas poza splotem i skalowaniem wyniósł 26,9 ms, czyli 48 % czasu W4, podczas gdy splot na 24 wątkach trwał 24,8 ms. W W1 ten sam przydział jest wliczony w czas splotu (strony pamięci są przydzielane przy pierwszym zapisie), więc nie widać go osobno.

W5 była najszybsza dla obrazów 4K i 8K. Dla 8K osiągnęła 15,6×, czyli więcej niż dla 4K, ponieważ stały narzut uruchomienia kerneli i synchronizacji z procesorem rozkłada się na więcej pikseli. Dla obrazu HD najszybsza była W3 (1,62 ms wobec 1,74 ms dla W5).

Maska 7×7 zmieniła kolejność wersji (rysunek 2). W2 z przyspieszeniem 13,7× była szybsza od W3 (12,6×) i W4 (11,6×), ponieważ splot rozdzielny wykonuje 28 zamiast 98 operacji na piksel. W5 osiągnęła 26,9×. Przesył danych i sterowanie z procesora nie zależą od maski, a czas obliczeń na karcie wzrósł tylko z 5,10 ms do 10,7 ms, choć liczba mnożeń wzrosła ponad 5 razy. Kernel karty jest ograniczony głównie odczytami z pamięci, a dodatkowe piksele otoczenia, potrzebne przy większej masce, czytane są przeważnie z pamięci podręcznej karty, bo czytały je już sąsiednie wątki.

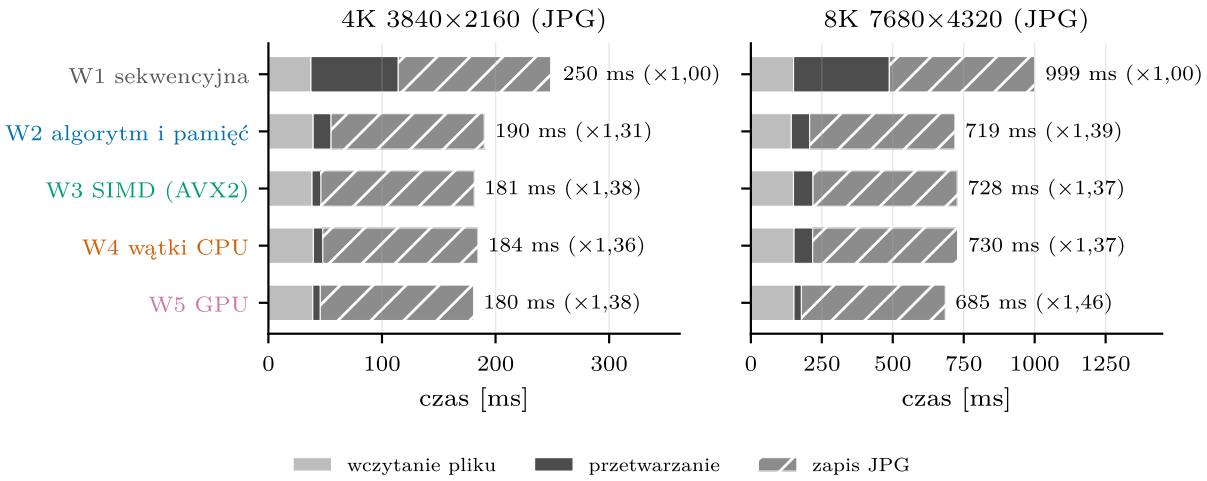
7.5 Rozkład czasu: wczytanie, algorytm, zapis

Rysunek 3 porównuje czas całego programu dla wszystkich wersji, a rysunki 4–8 pokazują dla każdej wersji, jak czas dzieli się na wczytanie obrazu, algorytm i zapis wyniku oraz na etapy samego algorytmu.

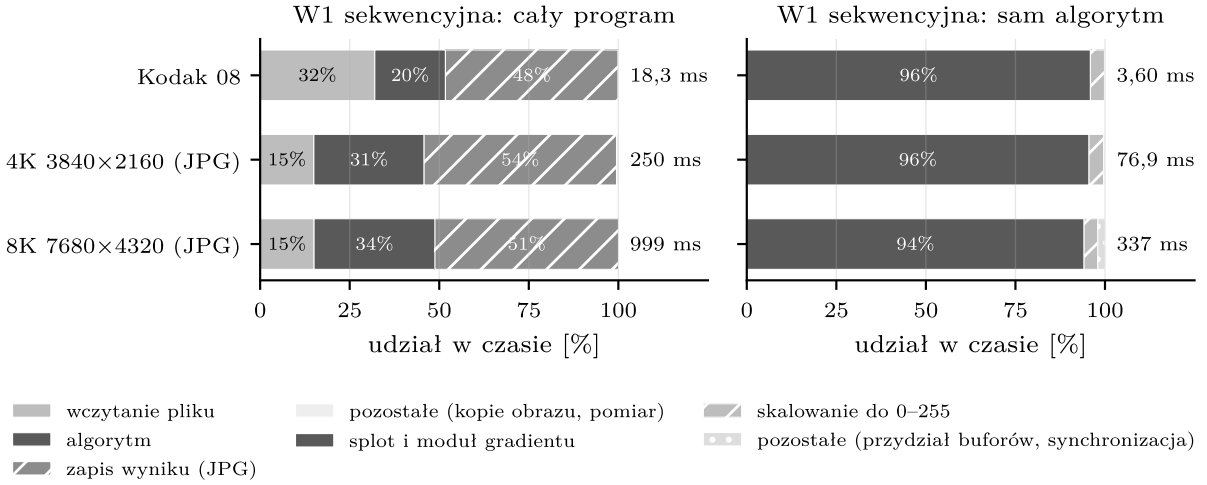
Tabela 4: Składniki czasu całego programu dla obrazu 4K (JPG) i przyspieszenie S samego algorytmu oraz całego programu względem W1.

Wersja	Wczyt. [ms]	Algor. [ms]	Zapis [ms]	Razem [ms]	S algor.	S całości
W1 sekwencyjna	37,5	76,9	134	250	1,00	1,00
W2 algorytm i pamięć	39,4	15,8	135	190	4,87	1,31
W3 SIMD (AVX2)	38,4	7,82	135	181	9,82	1,38
W4 wątki CPU	39,6	8,28	137	184	9,29	1,36
W5 GPU	39,1	6,64	135	180	11,6	1,38

Wczytanie i zapis obrazu okazały się dłuższe niż sam algorytm nawet w wersji W1. Dla obrazu 4K w formacie JPG W1 wczytywała plik 37,5 ms, przetwarzała 76,9 ms i zapisywała wynik 134 ms; algorytm stanowił tylko 30,8 % czasu całego programu, a sam zapis – 53,7 %. Zapis jest najdroższy, ponieważ użyty koder JPG (`stb_image_write`) jest prosty,



Rysunek 3: Czas całego programu: wczytanie pliku obrazu, algorytm i zapis wyniku w formacie JPG (maska Sobela 3×3); na końcu słupka czas całkowity i przyspieszenie całego programu względem W1.

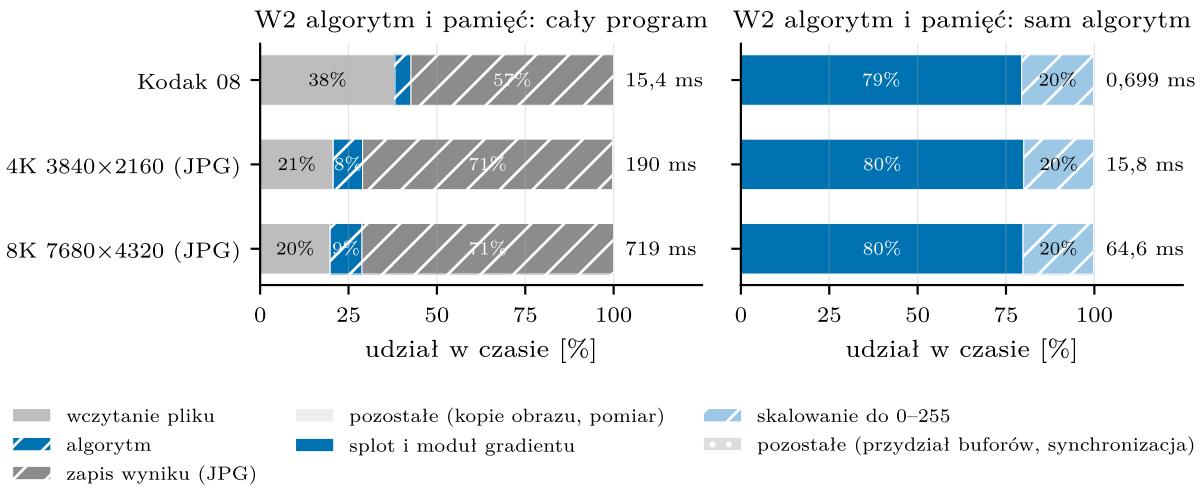


Rysunek 4: Rozkład czasu wersji W1: udział wczytania, algorytmu i zapisu w całym programie (lewy panel) oraz udział etapów w samym algorytmie (prawy panel); na końcu słupka czas całkowity. Sobel 3×3.

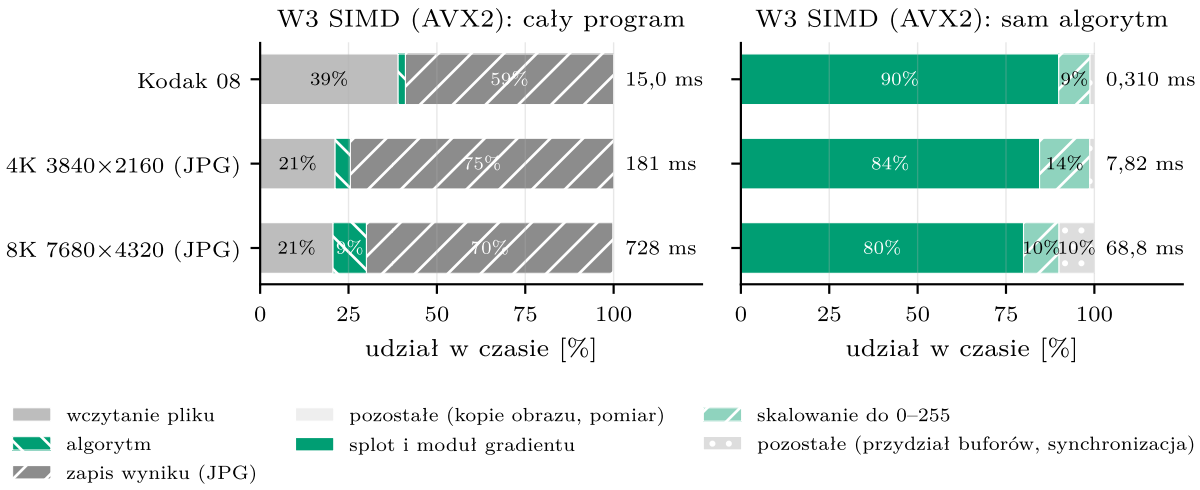
jednowątkowy i nie korzysta z instrukcji wektorowych. Prawdopodobnie wydłuża go też treść wyniku: obraz krawędzi ma dużo drobnych szczegółów, które dają więcej niezerowych współczynników do zakodowania niż gładkie zdjęcie. Tej hipotezy nie sprawdzano osobnym pomiarem.

Wczytanie i zapis nie zależą od wersji algorytmu, więc stanowią część sekwencyjną w rozumieniu prawa Amdahla (4). W rezultacie W3, W4 i W5, które przyspieszyły sam algorytm 9–12×, przyspieszyły cały program tylko 1,38–1,38× (tabela 4). W wersji W5 algorytm stanowił już tylko 3,7 % czasu całego programu, a wczytanie i zapis – 96,4 %. Dla maski 7×7, przy której algorytm W1 trwa dłużej, cały program z W2 przyspieszył 2,63×. Dalsze przyspieszanie samego algorytmu nie skróci więc odczuwalnie całego programu. Większy zysk dałoby przyspieszenie zapisu (punkt 7.11).

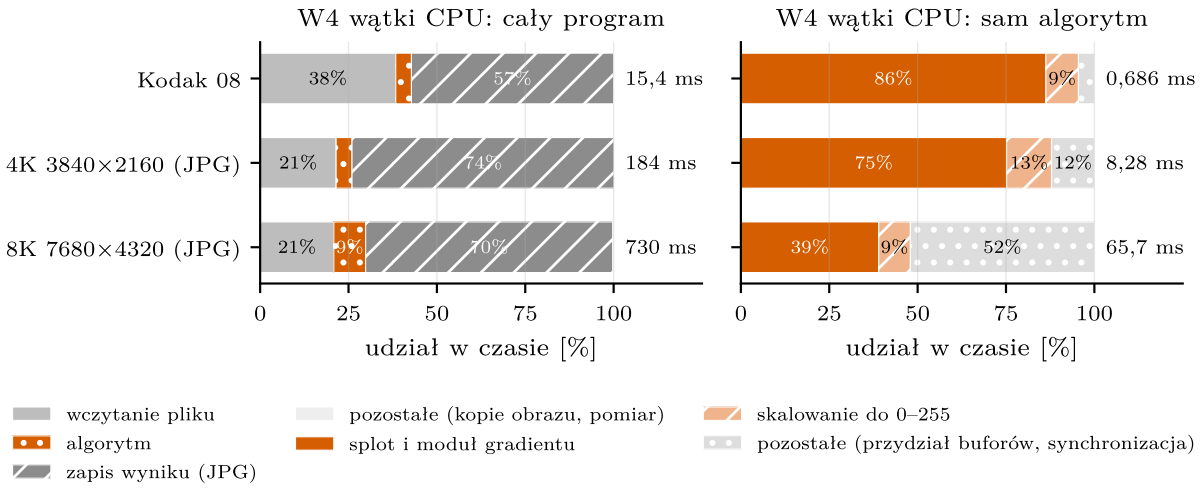
Rozkład czasu samego algorytmu (prawe panele rysunków 4–8) pokazuje, co ogranicza każdą wersję. W W1 i W2 prawie cały czas zajmuje splot. Skalowanie do 0–255 trwało



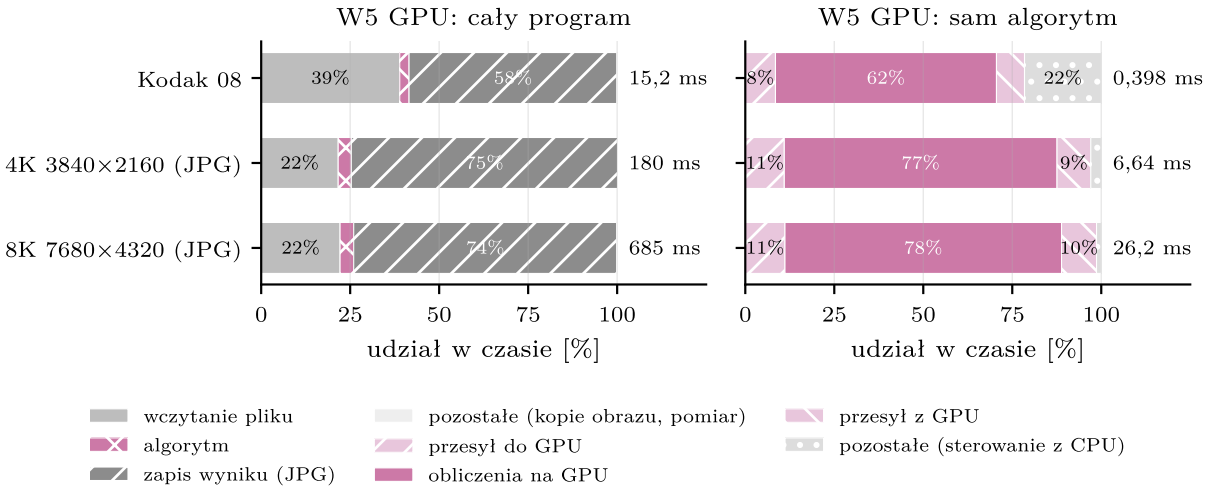
Rysunek 5: Rozkład czasu wersji W2 (jak na rysunku 4).



Rysunek 6: Rozkład czasu wersji W3 (jak na rysunku 4).



Rysunek 7: Rozkład czasu wersji W4 (jak na rysunku 4); „pozostałe” w algorytmie to przydział i zerowanie buforów oraz synchronizacja wątków.

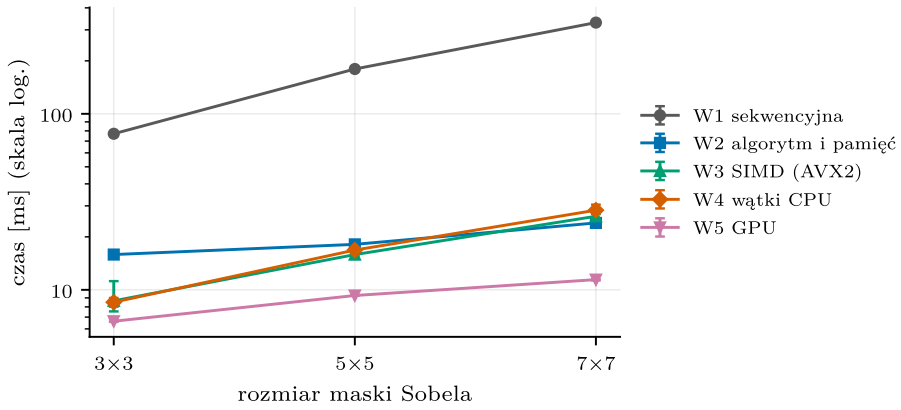


Rysunek 8: Rozkład czasu wersji W5: w algorytmie przesył obrazu na kartę, obliczenia na karcie, przesył wyniku i sterowanie z procesora. Inicjalizacja OpenCL odbywa się raz, poza pomiarem.

w W1 3,01 ms z 76,2 ms. W W2, która nie zmienia skalowania, ten sam etap (3,17 ms) stanowi już około 20 % czasu. W3 liczy wektorowo również skalowanie, które trwa w niej 1,21 ms. W W4 splot i skalowanie dzielą się pomiędzy wątki, ale przydział i zerowanie buforów nie, więc rośnie udział etapu „pozostałe”: dla 4K 13 %, a dla 8K już 48 %. W W5 czas przesłania obrazu na kartę (0,729 ms) i wyniku z powrotem (0,751 ms) jest kilkukrotnie krótszy od obliczeń na karcie (5,10 ms).

7.6 Wersja W2 i W3 – rozmiar maski

Wpływ rozmiaru maski na czas pokazano na rysunku 9 i w tabeli 5. Eksperyment dotyczy przede wszystkim W2, której przewaga wynika ze złożoności $O(k)$, i W3, która tej złożoności nie zmienia; pozostałe wersje pokazano dla porównania.



Rysunek 9: Czas samego algorytmu w funkcji rozmiaru maski Sobela dla wersji W1–W5 (obraz 4K); skala logarytmiczna.

Przejście z maski 3×3 na 7×7 wydłużyło czas W1 4,28-krotnie (z 77,0 ms do 330 ms). Liczba operacji rośnie przy tym $98/18 \approx 5{,}4$ razy. Czas rośnie wolniej, ponieważ część pracy nie zależy od maski: obliczenie modułu, zapis do bufora i skalowanie kosztują tyle

Tabela 5: Czas samego algorytmu [ms] dla różnych masek (obraz 4K).

Maska	W1 [ms]	W2 [ms]	W3 [ms]	W4 [ms]	W5 [ms]
Sobel 3×3	77,0	15,9	8,66	8,51	6,63
Sobel 5×5	180	18,1	15,9	16,8	9,28
Sobel 7×7	330	24,0	26,2	28,3	11,4
Prewitt 3×3	76,2	16,0	8,58	8,50	5,60
Scharr 3×3	76,9	16,1	8,96	8,54	7,78
Roberts 2×2	76,7	15,8	8,64	8,56	5,36

samo dla każdej maski. W W2 czas wzrósł tylko 1,51-krotnie, zgodnie z liniową złożonością splotu rozdzielnego, więc przyspieszenie W2 względem W1 wzrosło z 4,86× dla 3×3 przez 9,92× dla 5×5 do 13,7× dla 7×7. Maski 3×3 Prewitta i Scharra dały w W2 ten sam czas co Sobel. Nierozdzielna maska Robertsa, liczona splotem dwuwymiarowym, trwała 15,8 ms, czyli praktycznie tyle samo, ponieważ ma tylko 4 pola.

W3 nie zmienia złożoności $O(k^2)$, więc jej czas rośnie z rozmiarem maski podobnie jak w W1 (3,02-krotnie). Jej przyspieszenie względem W1 wzrosło przy tym z 8,90× do 12,6×. Przy większej masce większą część czasu zajmują mnożenia, które W3 wykonuje po 8 naraz, a mniejszą – operacje wykonywane raz na piksel. Dla maski 7×7 zmiana algorytmu (W2, 24,0 ms) dała więcej niż wektoryzacja (W3, 26,2 ms) i 24 wątki (W4, 28,3 ms), mimo że W2 działa na jednym rdzeniu i nie używa jawnie instrukcji wektorowych.

7.7 Wersja W4 – zarządzanie wątkami

7.7.1 Liczba wątków

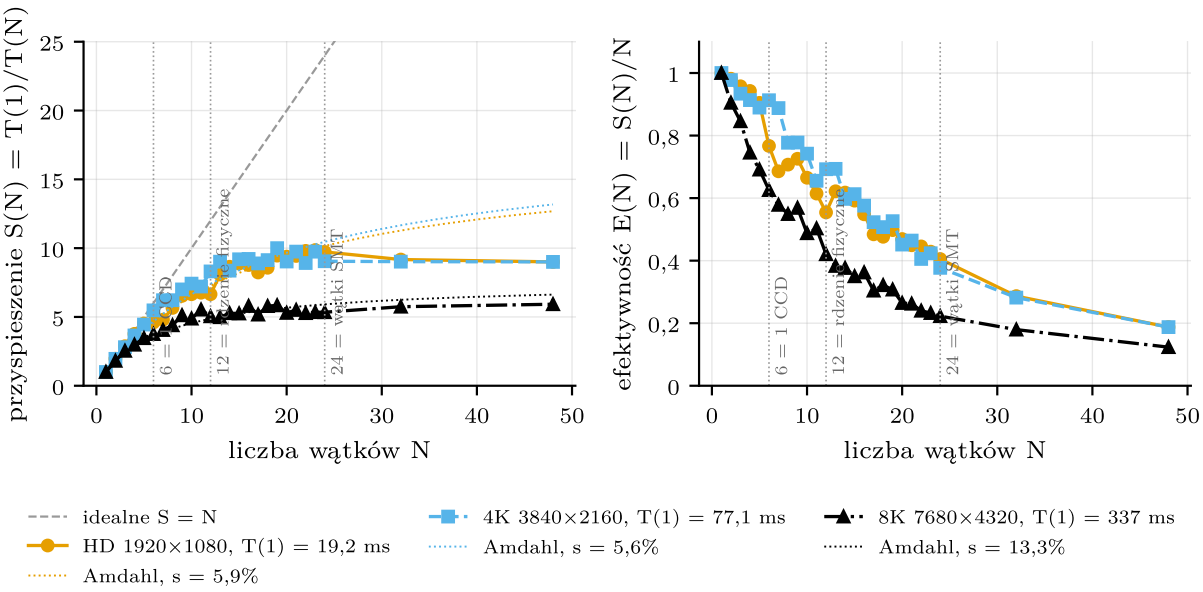
Przyspieszenie i efektywność W4 w funkcji liczby wątków pokazano na rysunku 10 i w tabeli 6.

Tabela 6: Skalowanie W4: czas na 1 wątku, przyspieszenie dla 12 i 24 wątków, największe przyspieszenie (liczba wątków) i dopasowany udział części sekwencyjnej s.

Obraz	T(1) [ms]	S(12)	S(24)	maks. S (N)	s [%]
HD 1920×1080	19,2	6,65	9,73	9,86 (23)	5,9
4K 3840×2160	77,1	8,31	9,05	10,00 (19)	5,6
8K 7680×4320	337	5,05	5,34	5,92 (48)	13,3

Dla obrazu 4K przyspieszenie na 2 wątkach wyniosło 1,96, na 12 wątkach (wszystkie rdzenie fizyczne) 8,31, a na 24 9,05. Największą wartość, 10,00, osiągnięto dla 19 wątków. Efektywność spadła z 98 % dla 2 wątków do 69 % dla 12 i 38 % dla 24. Prawo Amdahla dopasowane do pomiarów daje udział części sekwencyjnej $s = 5,6\,\%$ dla 4K i $s = 13,3\,\%$ dla 8K. Dla 8K przyspieszenie na 24 wątkach wyniosło tylko 5,34. Zgadza się to z rozkładem czasu z punktu 7.5: jednowątkowy przydział i zerowanie buforów rośnie liniowo z rozmiarem obrazu, a dla 8K dodatkowo dane nie mieszczą się w L3 i wątki konkurują o przepustowość pamięci.

Powyżej 12 wątków przyrost przyspieszenia dla 4K i 8K był niewielki (dla 4K z 8,31 do 9,05). Wyjątkiem był obraz HD (z 6,65 do 9,73); jego dane mieszczą się w pamięci L3, a przy 12 wątkach przetwarzanie trwa tylko 2,88 ms, więc różnica może wynikać także z



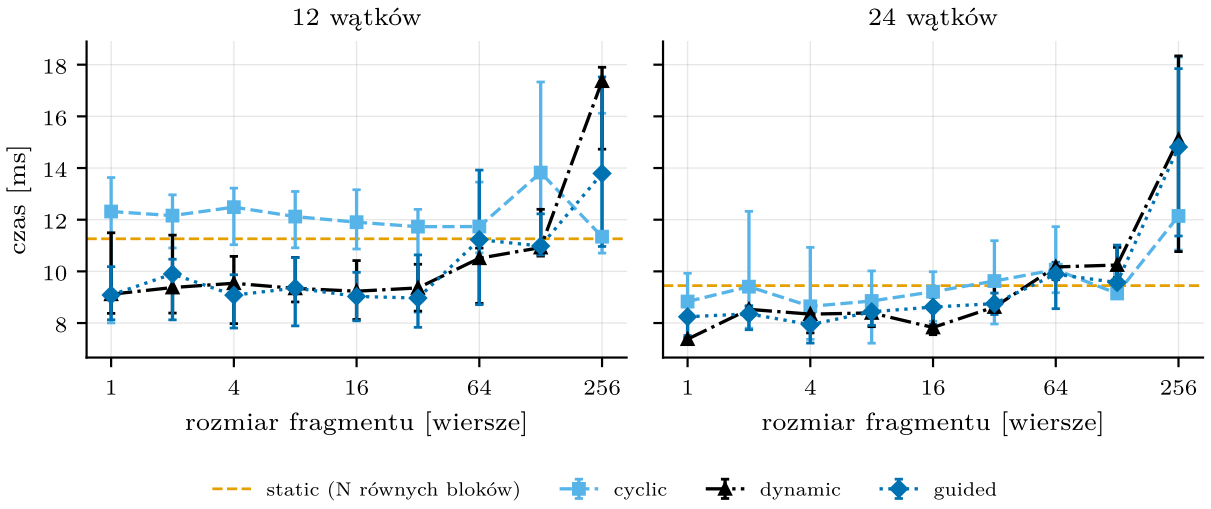
Rysunek 10: Przyspieszenie $S(N)$ i efektywność $E(N)$ wersji W4 względem tej samej wersji na 1 wątku (obrazy HD, 4K, 8K; harmonogram dynamic, fragment 16 wierszy). Linie kropkowane: prawo Amdahla dla dopasowanego s. Pionowe linie: 6 wątków (1 blok rdzeni), 12 (rdzenie fizyczne), 24 (wątki SMT).

rozrzutu pomiaru i nierównowagi obciążenia, których tu osobno nie badano. Drugi wątek sprzętowy (SMT) na tym samym rdzeniu współdzieli jednostki wykonawcze i pamięć podręczną L1/L2, więc pomaga tylko wtedy, gdy pierwszy wątek często czeka na pamięć. Więcej wątków niż rdzeni logicznych (32 i 48) nie zmieniło czasu w granicach rozrzutu (4K: 8,53 ms dla 24 i 8,57 ms dla 48 wątków). Przy harmonogramie *dynamic* wątek, który chwilowo czeka na rdzeń, po prostu pobiera mniej fragmentów, więc nadmiarowe wątki nie wydłużają etapu.

7.7.2 Strategia przydziału i rozmiar fragmentu

Rysunek 11 przedstawia czas W4 dla czterech strategii przydziału i rozmiarów fragmentu od 1 do 256 wierszy.

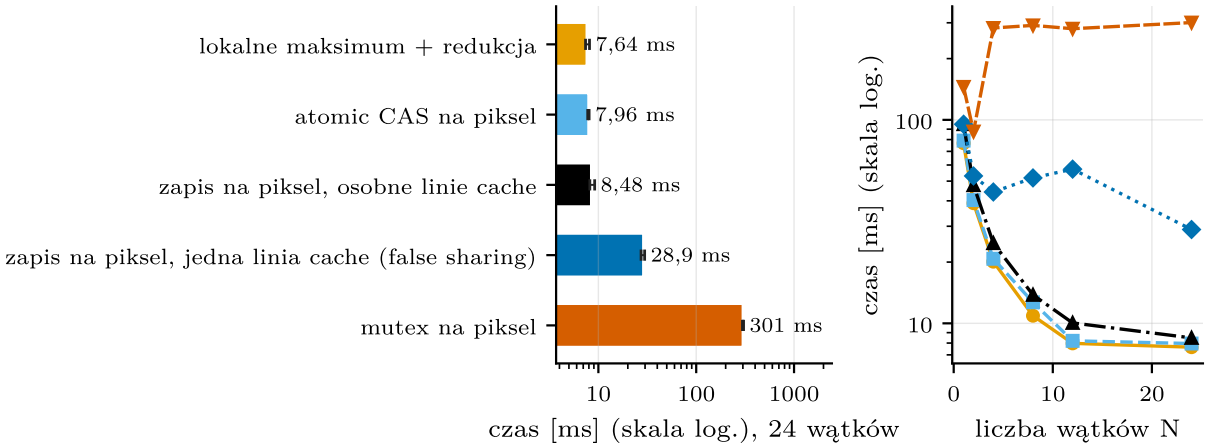
Dla 24 wątków najkrótszy czas, 7,38 ms, dała konfiguracja dynamic, 1 w. (harmonogram, rozmiar fragmentu), a najdłuższy, 15,1 ms, konfiguracja dynamic, 256 w.. Obraz 4K ma 2160 wierszy, więc fragmenty po 256 wierszy dają tylko 9 porcji pracy dla 24 wątków: 15 wątków nie ma nic do zrobienia, a nierównowaga (różnica czasu zakończenia pracy przez wątki) wyniosła 8,68 ms. Przy fragmencie 1 wiersza nierównowaga spadła do 0,064 ms. Koszt operacji atomowej na licznik fragmentów okazał się pomijalny. Jeden wiersz obrazu 4K jądro W1 liczy przez kilkadziesiąt mikrosekund, a operacja atomowa trwa kilkadziesiąt nanosekund, czyli około tysiąc razy krócej. Harmonogram *static* (jeden blok na wątek) był wolniejszy (9,45 ms) z nierównowagą 4,29 ms, ponieważ nie kompensuje wątków spowolnionych przez system lub przez współdzielenie rdzenia w SMT. Dla 12 wątków najlepsza była konfiguracja guided, 32 w. (8,97 ms), ale różnice między *dynamic* i *guided* przy fragmentach do 32 wierszy mieściły się w rozrzucie pomiaru.



Rysunek 11: Czas W4 w funkcji rozmiaru fragmentu dla czterech strategii przydziału (obraz 4K, 12 i 24 wątki). Strategia static nie używa rozmiaru fragmentu.

7.7.3 Synchronizacja przy wyznaczaniu maksimum

Rysunek 12 porównuje pięć sposobów wyznaczania maksimum globalnego.



Rysunek 12: Czas W4 dla pięciu sposobów wyznaczania maksimum globalnego: dla 24 wątków (lewy panel) i w funkcji liczby wątków (prawy panel, te same kolory). Obraz 4K, skala logarytmiczna.

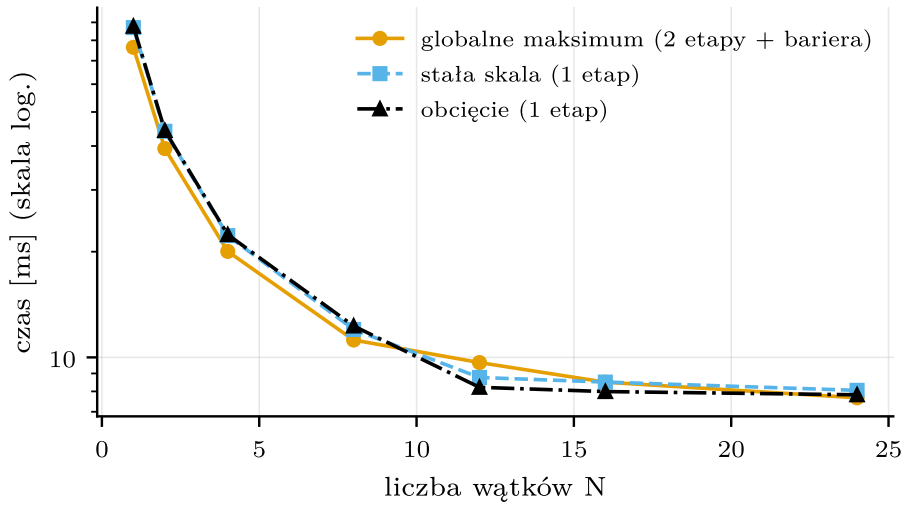
Lokalne maksimum każdego wątku z redukcją po barierze dało 7,64 ms na 24 wątkach. Wspólna zmienna atomowa była tylko 1,04× wolniejsza (7,96 ms). Wątek odczytuje ją przy każdym pikselu, ale zapisuje tylko wtedy, gdy znajdzie większą wartość. Po kilku pierwszych wierszach takie zapisy zdarzają się rzadko, więc linia pamięci podręcznej z tą zmienną przez większość czasu jest tylko odczytywana i może być jednocześnie przechowywana przez wszystkie rdzenie. Blokada `std::mutex` przy każdym pikselu wydłużyła czas do 301 ms (39,4×). Już na jednym wątku, bez rywalizacji, kosztowała 145 ms wobec 76,8 ms, a od 4 wątków czas przestał zależeć od liczby wątków (283–301 ms), bo sekcja krytyczna szeregowała wszystkie wątki.

Zapis bieżącego maksimum do własnej komórki wątku przy każdym pikselu pokazał wpływ fałszywego współdzielenia. Gdy komórki wątków leżały w osobnych liniach pamięci

podręcznej, czas na 24 wątkach wyniósł 8,48 ms. Gdy leżały obok siebie w jednej linii – 28,9 ms (3,79× dłużej niż wersja lokalna), mimo że wątki nigdy nie zapisywały tej samej zmiennej. Każdy zapis unieważnia linię w pamięciach podręcznych pozostałych rdzeni. Na jednym wątku obie wersje trwały tyle samo (95,2 ms), co potwierdza, że różnica wynika wyłącznie z protokołu spójności pamięci podręcznych.

7.7.4 Bariera wynikająca ze skalowania

Rysunek 13 porównuje skalowanie względem maksimum ze skalowaniem stałym i obcięciem.



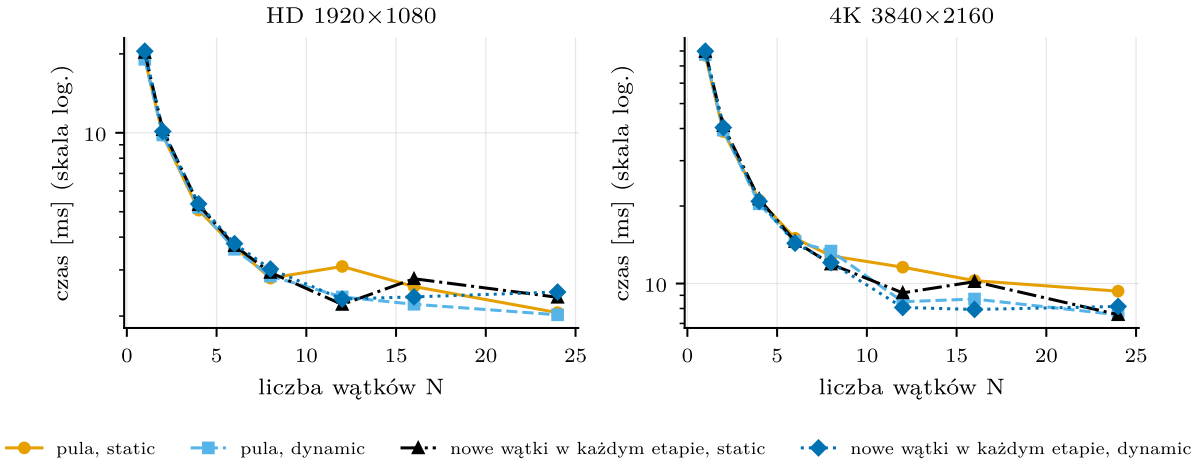
Rysunek 13: Czas W4 dla skalowania względem maksimum (2 etapy i bariera), stałej skali i obcięcia (1 etap); obraz 4K.

Usunięcie bariery i drugiego przebiegu nie skróciło czasu. Na 24 wątkach skalowanie względem maksimum trwało 7,67 ms, a skalowanie stałe i obcięcie odpowiednio 8,06 ms i 7,82 ms. Na jednym wątku wersje jednoetapowe były nawet wolniejsze (87,0 ms i 87,7 ms wobec 76,4 ms). W wersjach jednoetapowych jądro W1 zamienia moduł na bajt wewnątrz pętli splotu, co wydłuża najczęściej wykonywaną pętlę. Przypuszczalnie utrudnia to kompilatorowi jej optymalizację, ale przyczyny tej różnicy nie badano dokładniej. Dla W4 wynika z tego, że bariera między etapami kosztuje mniej niż wynosi rozrzut pomiaru. Wymaganie skalowania względem maksimum nie ogranicza więc skalowania wielowątkowego.

7.7.5 Pula wątków a tworzenie wątków

Rysunek 14 porównuje pulę wątków z tworzeniem nowych wątków w każdym etapie.

Dla obrazu HD i 24 wątków z harmonogramem *dynamic* pula dała 2,02 ms, a tworzenie wątków 2,47 ms. Przy dwóch etapach i 24 wątkach jedno wywołanie tworzy i kończy 48 wątków, więc różnica odpowiada około 10 μs na wątek. Dla obrazu 4K, którego przetwarzanie trwa kilka razy dłużej, różnica znikała w rozrzucie pomiaru (7,56 ms wobec 8,15 ms), a dla harmonogramu *static* pula była nawet wolniejsza (9,34 ms wobec 7,57 ms). Uśpione wątki puli są budzone przez zmienną warunkową, a obudzenie wątku przez system operacyjny zajmuje czas porównywalny z utworzeniem nowego. Pula daje

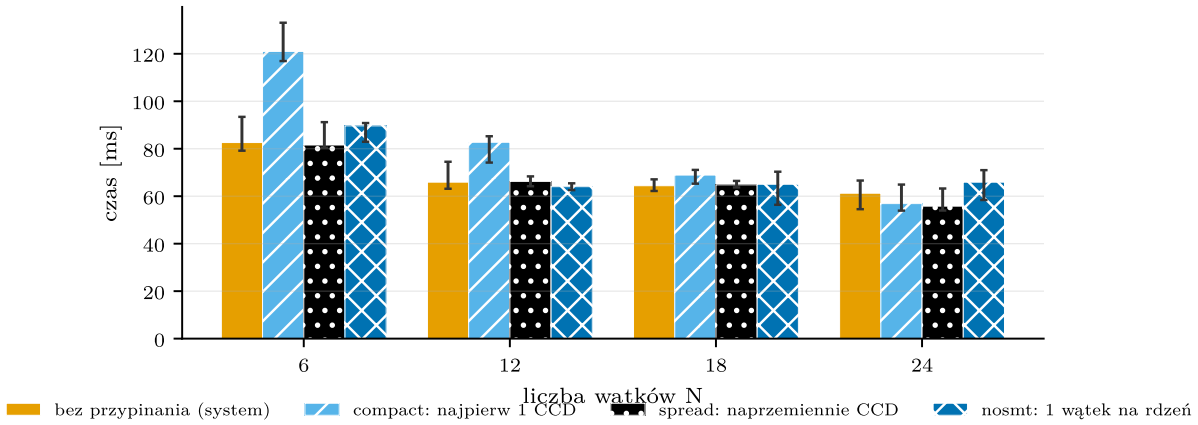


Rysunek 14: Czas W4 z pulą wątków i z nowymi wątkami tworzonymi w każdym etapie (harmonogramy static i dynamic, obrazy HD i 4K).

więc zauważalny zysk dopiero przy krótkich zadaniach, takich jak przetwarzanie obrazu HD.

7.7.6 Przypisanie wątków do rdzeni

Rysunek 15 przedstawia wpływ przypisania wątków do rdzeni.

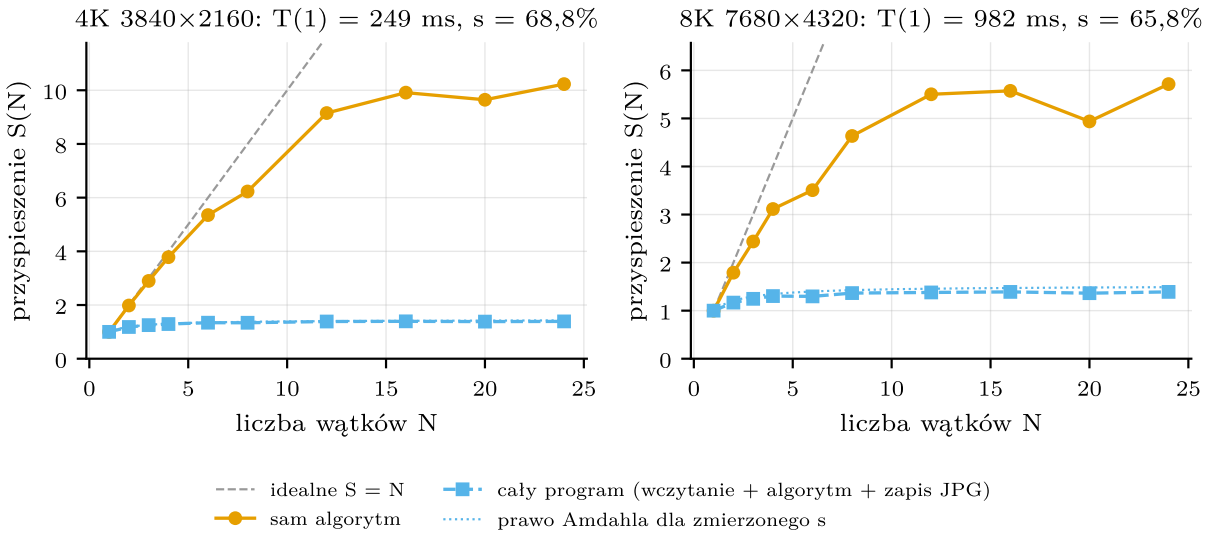


Rysunek 15: Czas W4 dla czterech strategii przypisania wątków do rdzeni (obraz 8K).

Największą różnicę dało przypisanie przy 6 wątkach. Strategia „najpierw jeden blok” umieszcza je na 3 rdzeniach fizycznych po 2 wątki SMT i dała 121 ms. Rozłożenie na oba bloki rdzeni dało 81,6 ms, a pozostawienie decyzji systemowi – 82,6 ms, ponieważ planista systemu sam unika umieszczania dwóch wątków na jednym rdzeniu, gdy są wolne rdzenie. Dla 12 wątków strategia „jeden wątek na rdzeń” (64,0 ms) i brak przypisania (65,9 ms) dały praktycznie ten sam czas, a „najpierw jeden blok” znowu była najwolniejsza (82,7 ms). Dla 24 wątków, gdy zajęte są wszystkie wątki sprzętowe, różnice między strategiami (55,8–61,3 ms) były porównywalne z rozrzutem pomiaru. Ręczne przypisanie wątków nie dało więc zysku względem planisty systemu, a złe przypisanie wydłużyło czas nawet 1,5-krotnie.

7.7.7 Cały program w funkcji liczby wątków

Rysunek 16 porównuje przyspieszenie samego algorytmu i całego programu.



Rysunek 16: Przyspieszenie samego algorytmu i całego programu (wczytanie JPG, algorytm W4, zapis JPG) w funkcji liczby wątków; linia kropkowana: prawo Amdahla dla zmierzonego udziału wczytania i zapisu.

Dla obrazu 4K wczytanie (36,5 ms) i zapis (135 ms) stanowiły $s = 68,8\,\%$ czasu całego programu na jednym wątku. Prawo Amdahla ogranicza wtedy przyspieszenie całego programu do $1/s \approx 1,45$ przy dowolnej liczbie wątków. Zmierzone przyspieszenie całego programu na 24 wątkach wyniosło 1,39, przy przyspieszeniu samego algorytmu 10,2. Dla obrazu 8K przyspieszenie całego programu było takie samo (1,39 wobec granicy 1,52). Pomiar zgadza się więc z prawem Amdahla. Zwiększanie liczby wątków w algorytmie nie skróci programu, którego większą część zajmują sekwencyjne operacje na plikach.

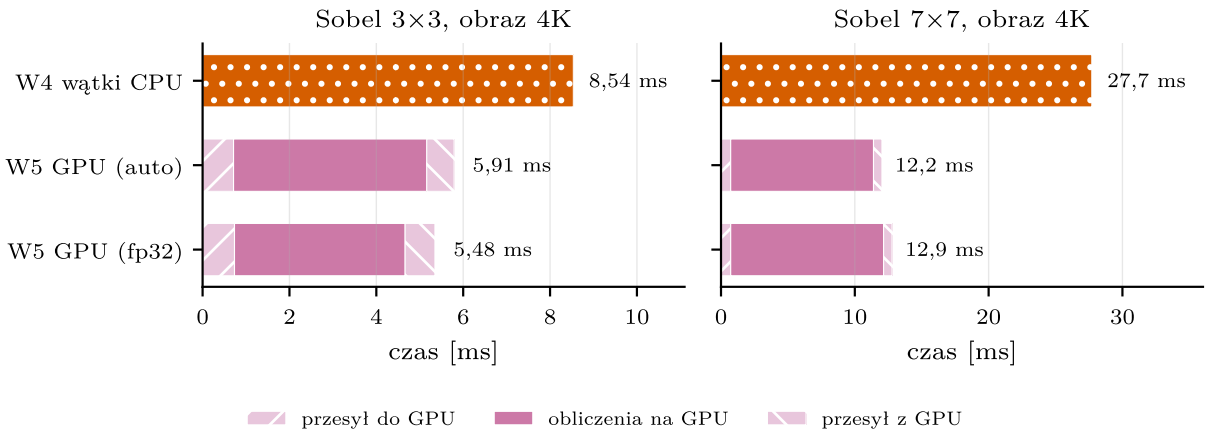
7.8 Wersja W5 – karta graficzna

Etapy W5 dla obrazu 4K pokazano na rysunku 17, zależność od rozmiaru obrazu – na rysunku 18, a wpływ liczby wątków GPU – na rysunku 19.

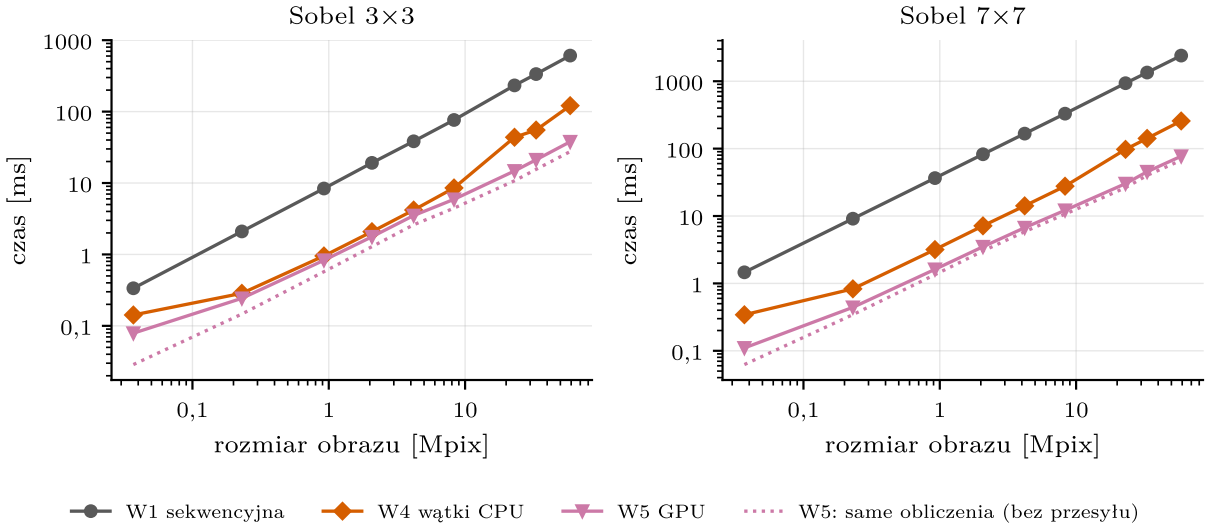
Dla obrazu 4K i maski 3×3 W5 potrzebowała 5,91 ms: przesłanie obrazu na kartę 0,713 ms, obliczenia 4,45 ms i przesłanie wyniku 0,630 ms. Przesył przez magistralę PCIe zajął więc około 20 % czasu, a resztę stanowiły obliczenia na karcie i sterowanie z procesora (uruchomienie kerneli, odczyt maksimum, oczekiwanie na zakończenie). W4 na 24 wątkach potrzebowała w tym samym eksperymencie 8,54 ms, a dla 8K – 55,2 ms wobec 21,2 ms dla W5. Naiwny kernel W5 wykonuje te same obliczenia co W1, ale bez części sekwencyjnej W4, ponieważ bufory na karcie przydzielono raz, przed pomiarem.

Wymuszenie pojedynczej precyzji skróciło obliczenia z 4,45 ms do 3,93 ms dla maski 3×3 i z 10,7 ms do 11,4 ms dla 7×7, kosztem różnicy wyniku do 1 poziomu szarości. Zysk jest niewielki, ponieważ tryb dokładny liczy w podwójnej precyzji tylko piksele o dużym module gradientu, których na typowym obrazie jest mało.

Liczba wątków GPU wyraźnie wpływała na czas obliczeń (rysunek 19). Dla 4K jeden wątek na piksel w grupach 32×32 (8 355 840 wątków) był najwolniejszy: 6,21 ms. Najszybsza konfiguracja, 8×8, 16 px/wątek, uruchamiała 518 400 wątków i liczyła 2,62 ms,



Rysunek 17: Czas W5 z podziałem na przesył obrazu na kartę, obliczenia i przesył wyniku, wobec W4 (obraz 4K, maski 3×3 i 7×7); auto – obliczenia dokładne, fp32 – pojedyncza precyzja.



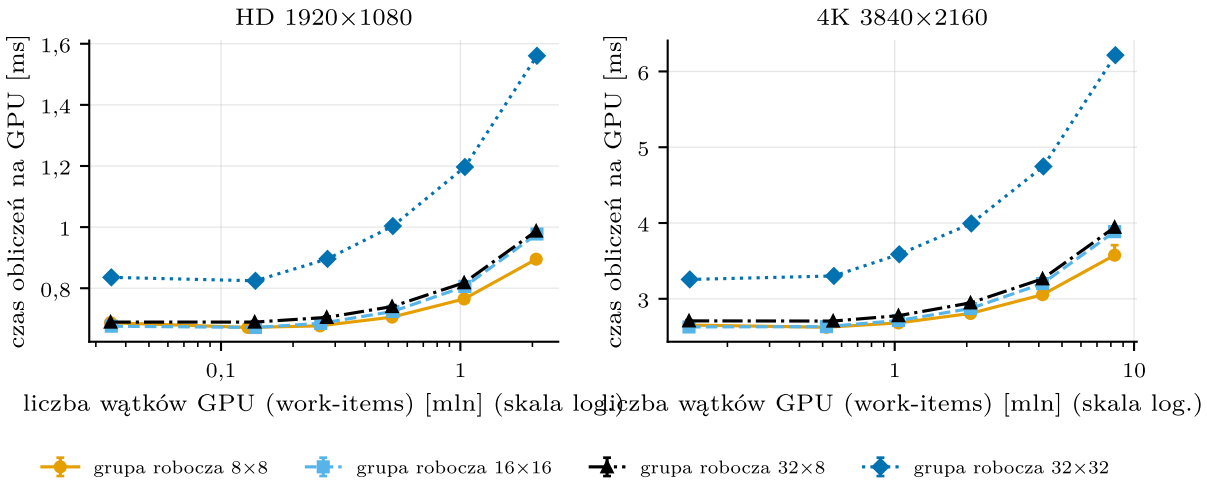
Rysunek 18: Czas W1, W4 i W5 w funkcji rozmiaru obrazu; linia kropkowana: same obliczenia na karcie (bez przesyłu).

czyli 2,4× krócej. Zmniejszenie liczby wątków przy tej samej pracy pomaga, ponieważ każdy wątek i każda grupa robocza mają stały koszt (uruchomienie, redukcja maksimum w pamięci lokalnej, operacja atomowa grupy), a 768 rdzeni karty i tak jest w pełni zajętych przy kilkuset tysiącach wątków. Grupa 32×32 (1024 wątki) była najwolniejsza we wszystkich konfiguracjach. Duża grupa ogranicza liczbę grup aktywnych jednocześnie na multiprocesorze i wydłuża redukcję maksimum w pamięci lokalnej. Powyżej 16 pikseli na wątek czas przestał maleć: wątków było wtedy już tylko kilkaset tysięcy, a dalsze ich zmniejszanie nie zmniejszało stałych kosztów w zauważalny sposób.

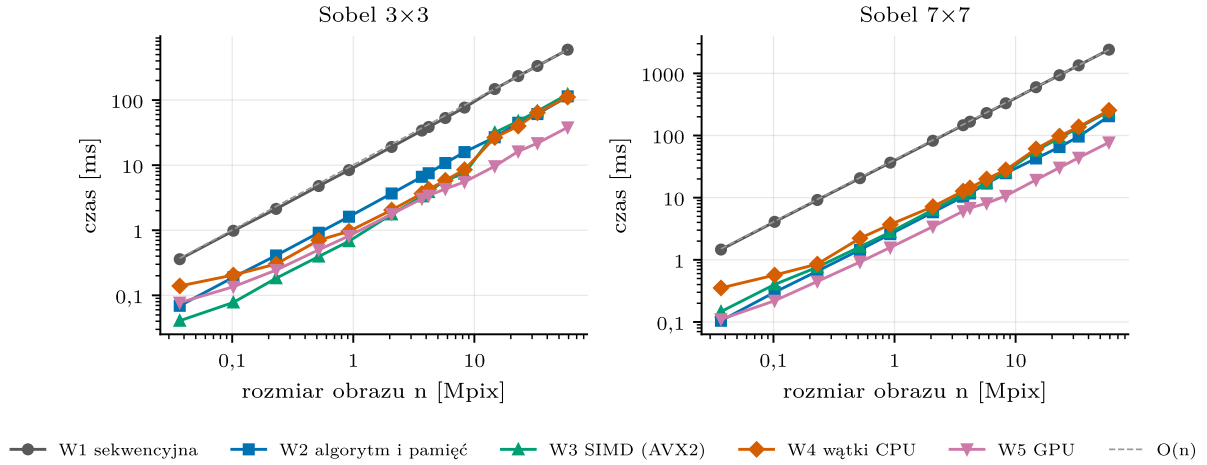
7.9 Rozmiar danych wejściowych

Zależność czasu, przyspieszenia i przepustowości od rozmiaru obrazu przedstawiono na rysunkach 20–22 i w tabelach 7–8.

Wszystkie wersje były szybsze od W1 już dla najmniejszego obrazu (256×144,



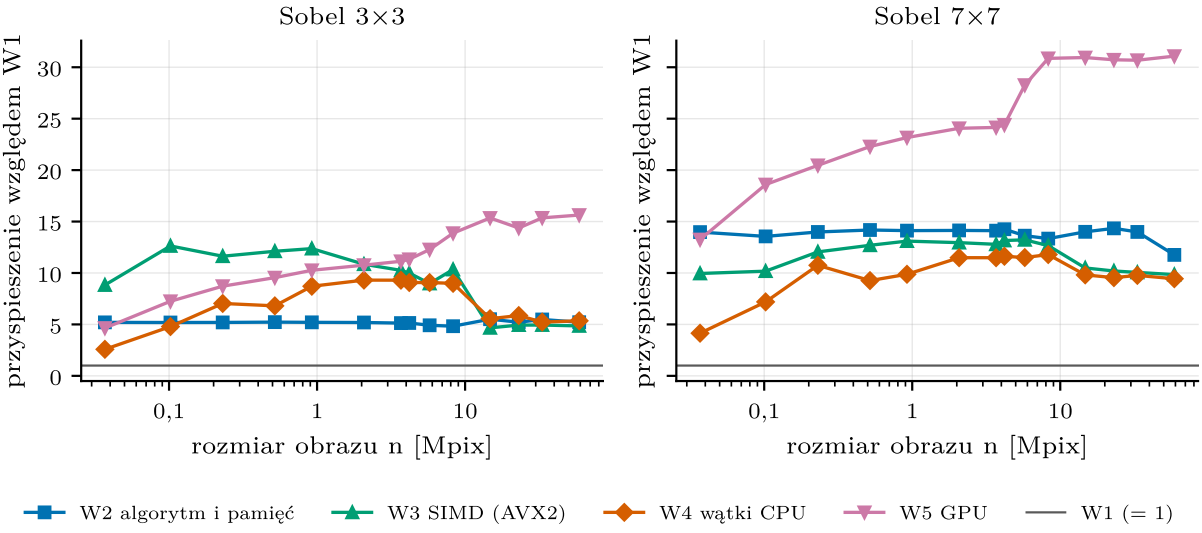
Rysunek 19: Czas obliczeń na karcie w funkcji liczby uruchomionych wątków GPU (liczba pikseli na wątek od 1 do 64) dla czterech rozmiarów grupy roboczej (obrazy HD i 4K, Sobel 3×3).



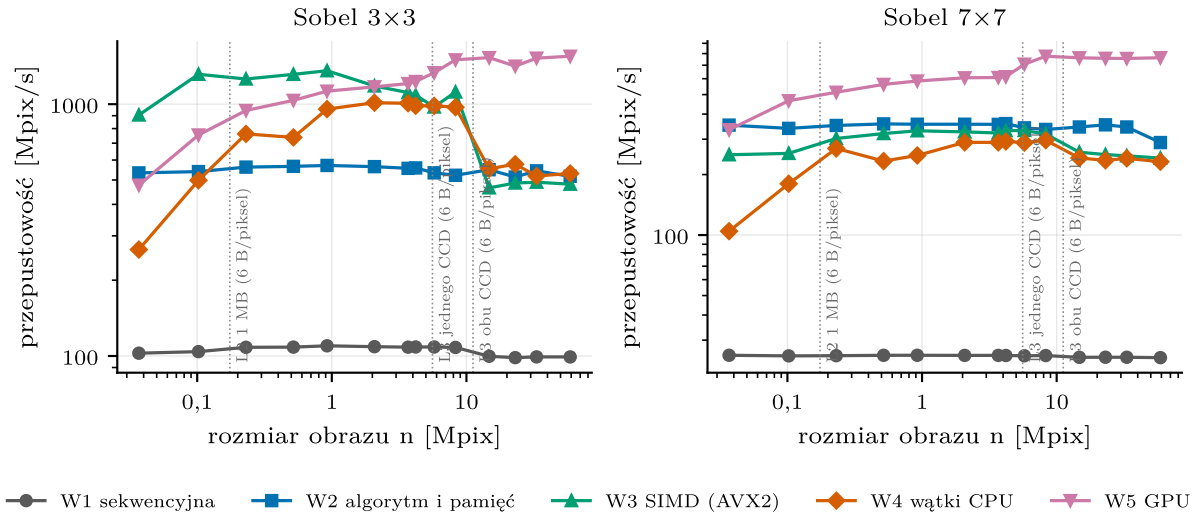
Rysunek 20: Czas samego algorytmu w funkcji liczby pikseli dla W1–W5 (skala logarytmiczna na obu osiach); linia przerywana: zależność liniowa $O(n)$.

Tabela 7: Czas [ms] dla obrazów o tej samej liczbie pikseli (≈4,2 Mpix) i różnych proporcjach.

Wersja	2731×1536 [ms]	2048×2048 [ms]	16384×256 [ms]
W1 sekwencyjna	38,6	38,6	38,6
W2 algorytm i pamięć	7,51	7,38	7,68
W3 SIMD (AVX2)	3,86	3,37	3,83
W4 wątki CPU	4,26	3,87	5,26
W5 GPU	3,41	2,81	2,90



Rysunek 21: Przyspieszenie wersji W2–W5 względem W1 w funkcji rozmiaru obrazu.



Rysunek 22: Przepustowość [Mpix/s] w funkcji rozmiaru obrazu; pionowe linie: rozmiar, przy którym dane (6 B/piksel) przekraczają pamięć L2 rdzenia, L3 jednego bloku rdzeni i L3 obu bloków.

Tabela 8: Kontrola: czas [ms] dla zdjęcia i obrazu syntetycznego tego samego rozmiaru.

Wersja	Rozmiar	Zdjęcie [ms]	Syntetyczny [ms]	Różnica [%]
W1 sekwencyjna	1920×1080	19,0	19,0	-0,0
W1 sekwencyjna	3840×2160	76,2	76,5	-0,3
W3 SIMD (AVX2)	1920×1080	1,68	1,65	1,8
W3 SIMD (AVX2)	3840×2160	8,61	7,40	16,4
W4 wątki CPU	1920×1080	2,05	1,93	6,2
W4 wątki CPU	3840×2160	8,38	8,47	-1,0

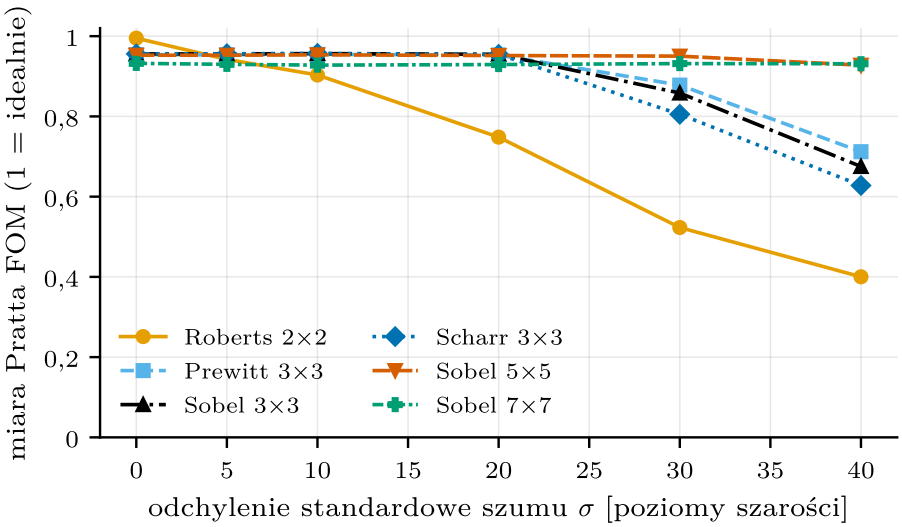
0,04 Mpix), ale zysk zależał od rozmiaru w różny sposób. Przyspieszenie W2 było stałe (5,20× dla 0,04 Mpix, 5,21× dla 59 Mpix), bo W2 nie ma stałych narzutów i pracuje na buforze wielkości kilku wierszy. W4 dla najmniejszego obrazu osiągnęła tylko 2,58× (0,139 ms): przy 144 wierszach i fragmencie 16 wierszy pracy jest tylko 9 porcji dla 24 wątków, a obudzenie wątków puli trwa porównywalnie z całym przetwarzaniem. Najwyższe przyspieszenie, 9,31×, W4 osiągnęła dla obrazów średniej wielkości, a dla 59 Mpix spadło ono do 5,35×. W5 zyskiwała z rozmiarem (do 15,6×), bo stałe koszty uruchomienia kerneli i synchronizacji rozkładają się na więcej pikseli. Do rozmiaru HD włącznie W3 była co najmniej tak szybka jak W5 (dla 1920 × 1080: 1,76 ms wobec 1,77 ms), a od 2560 × 1440 szybsza była W5 (3,05 ms wobec 3,32 ms).

Najwyraźniejszy wpływ pamięci podręcznej widać w W3 (rysunek 22). Między obrazem 3840 × 2160 (8,3 Mpix, 7,42 ms) a 5120 × 2880 (14,7 Mpix, 31,7 ms) liczba pikseli wzrosła 1,8 raza, a czas – ponad 4 razy. Przy 6 B/piksel dane obrazu 4K (około 50 MB) częściowo mieszczą się w pamięci L3, a obrazu 5K (około 88 MB) już nie, i jednowątkowe jądro wektorowe zaczyna czekać na pamięć operacyjną. Przyspieszenie W3 spadło z 12,4× dla 1280 × 720 do 4,85× dla 59 Mpix.

Proporcje obrazu przy tej samej liczbie pikseli nie miały znaczenia dla W1 (38,6 ms dla każdego kształtu). W W4 obraz 16384 × 256 był wyraźnie wolniejszy (5,26 ms wobec 3,87 ms dla 2048 × 2048), ponieważ 256 wierszy w fragmentach po 16 daje tylko 16 porcji pracy dla 24 wątków. Treść obrazu (zdjęcie lub obraz syntetyczny) nie wpływała na czas W1 i W4; największa różnica, 16,4 %, wystąpiła dla W3 i obrazu 4K, której czas dla tego rozmiaru miał też największy rozrzut, więc nie przypisano jej treści obrazu.

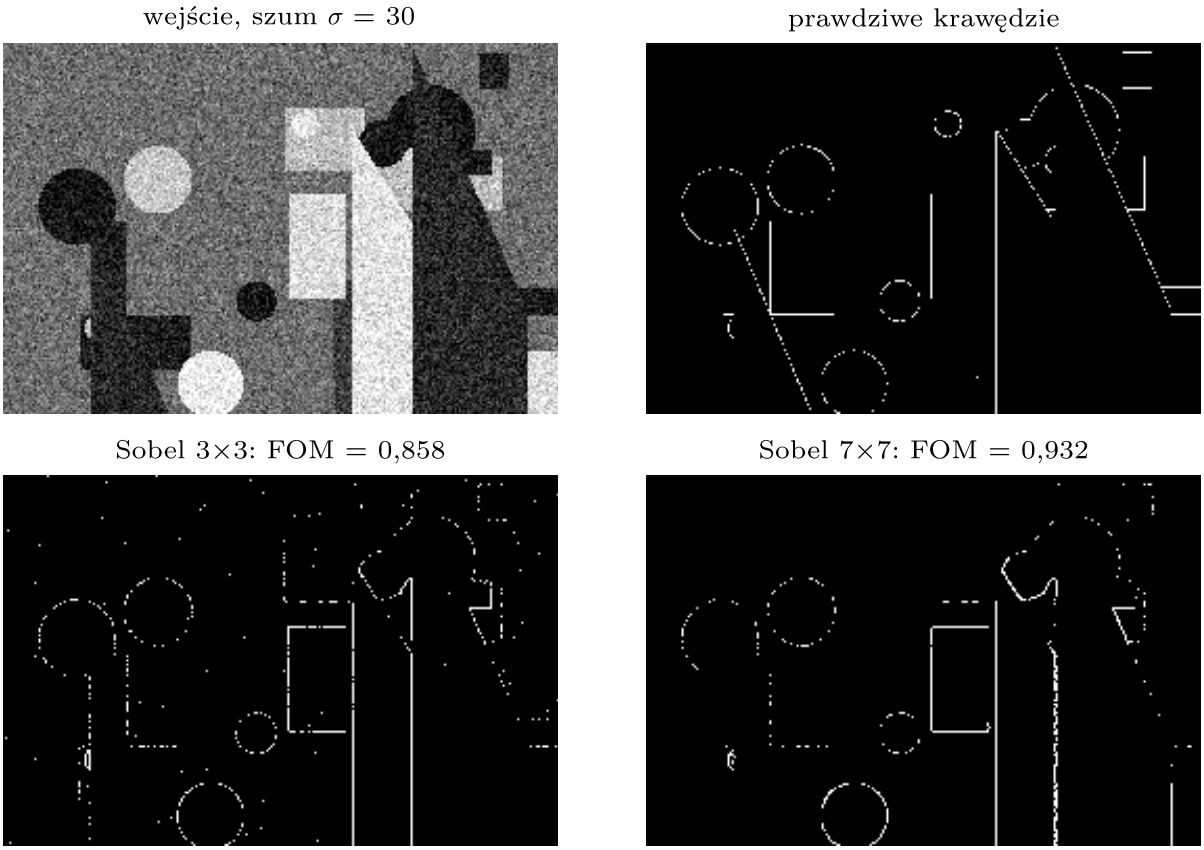
7.10 Jakość wykrywania krawędzi a rozmiar maski

Wynik porównania masek miarą Pratta przedstawiono na rysunku 23 i w tabeli 9, a przykład – na rysunku 24.



Rysunek 23: Miara Pratta (FOM) w funkcji poziomu szumu dla sześciu masek; obraz syntetyczny 768 × 512 o znanych krawędziach, próg najlepszy dla każdej maski i poziomu szumu.

Odpowiedź na pytanie, czy większa maska daje lepszy wynik, zależy od poziomu szumu. Na obrazie bez szumu najlepsza była najmniejsza maska, Roberts 2×2 (F =



Rysunek 24: Przykład: obraz z szumem $\sigma = 30$, prawdziwe krawędzie oraz krawędzie wykryte maskami Sobela 3×3 i 7×7 przy najlepszym progu.

Tabela 9: Miara Pratta dla masek i poziomów szumu.

Maska	$\sigma = 0$	$\sigma = 5$	$\sigma = 10$	$\sigma = 20$	$\sigma = 30$	$\sigma = 40$
Roberts 2×2	0,995	0,942	0,903	0,748	0,523	0,400
Prewitt 3×3	0,954	0,954	0,955	0,953	0,878	0,712
Sobel 3×3	0,956	0,956	0,957	0,955	0,858	0,675
Scharr 3×3	0,956	0,956	0,957	0,955	0,805	0,628
Sobel 5×5	0,953	0,953	0,953	0,952	0,950	0,928
Sobel 7×7	0,932	0,930	0,928	0,929	0,932	0,931

$0,995$), a Sobel 7×7 najgorsza ($F = 0,932$). Duża maska wygładza obraz w szerszym otoczeniu, więc wykryta krawędź ma kilka pikseli szerokości, a miara Pratta karze piksele odsunięte od prawdziwej krawędzi. Przy szumie o $\sigma = 20$ najlepsza była Sobel 3×3, przy $\sigma = 30$ – Sobel 5×5, a przy $\sigma = 40$ – Sobel 7×7. Wynik maski 7×7 prawie nie zależał od szumu (0,932–0,931), natomiast Sobel 3×3 spadł z 0,956 do 0,675, a Roberts – do 0,400. Uśrednianie w większym otoczeniu tłumi szum, ale kosztem dokładności położenia krawędzi.

Wybór maski jest więc kompromisem między jakością a czasem: Sobel 7×7 jest w W1 4,28× wolniejszy od 3×3, ale w W2 tylko 1,51×, a w W5 1,72×. Przy zaszumionych obrazach większa maska z wersją W2 lub W5 daje lepszy wynik przy niewielkim koszcie czasu. Wynik dotyczy jednego obrazu syntetycznego z szumem gaussowskim i progu dobranego optymalnie dla każdego przypadku. Na zdjęciach rzeczywistych, dla których

progu nie zna się z góry, wyniki mogą być inne.

7.11 Propozycje dalszych przyspieszeń (AI)

Zgodnie z wymaganiami projektu do zaproponowania dalszych ulepszeń użyto modelu sztucznej inteligencji – Claude (Anthropic), uruchomionego w narzędziu Claude Code z dostępem do kodu projektu i wyników pomiarów. Model miał wskazać przyspieszenia, których projekt jeszcze nie zawiera, i uzasadnić je zmierzonymi danymi. Należy zaznaczyć, że ten sam model pomagał wcześniej w implementacji, więc propozycje nie są niezależną oceną z zewnątrz. Żadnej z propozycji nie wdrożono ani nie zmierzono; podane zyski są szacunkami wynikającymi z pomiarów opisanych wyżej.

1. **Szybszy zapis wyniku.** Zapis JPG stanowił 53,7 % czasu całego programu W1 i 74,8 % czasu W5. Zastąpienie kodera `stb` biblioteką libjpeg-turbo (koder z instrukcjami SIMD) albo podział obrazu na pasy kodowane równolegle (znaczniki restartu JPEG) skróciłby największą część sekwencyjną. Jest to jedyna z propozycji, która może wyraźnie przyspieszyć cały program, bo zgodnie z prawem Amdahla żadne przyspieszenie samego algorytmu nie przekroczy $1/s \approx 1,45$.

2. **Przydział buforów poza pomiarem w W4.** Jednowątkowy przydział i zerowanie buforów stanowiły 48 % czasu W4 dla 8K. Bufory przydzielone raz (jak w W2) albo zerowane równolegle przez wątki puli usunęłyby tę część sekwencyjną; według prawa Amdahla dla $s = 13,3\,\%$ ograniczenie przyspieszenia wzrosłoby z około 7,5 do kilkunastu.

3. **Połączenie mechanizmów.** Wersje celowo nie były łączone. Połączenie splotu rozdzielnego (W2), instrukcji wektorowych (W3) i wątków (W4) powinno dać przyspieszenia, które się mnożą, aż do granicy przepustowości pamięci.

4. **AVX-512.** Procesor obsługuje 512-bitowe instrukcje wektorowe, które przetwarzają 16 zamiast 8 liczb 32-bitowych na instrukcję. Zysk byłby widoczny tylko dla obrazów mieszczących się w pamięci podręcznej (do 4K), bo dla większych W3 jest już ograniczona przepustowością pamięci (punkt 7.9).

5. **Kafelkowanie.** Przetwarzanie obrazu w poziomych pasach mieszczących się w pamięci L2 rdzenia (1 MB), z oboma etapami (splot i skalowanie) wykonywanymi dla pasa przed przejściem do następnego, zmniejszyłoby spadek przepustowości dla obrazów większych niż 4K.

6. **Aktywne oczekiwanie w puli wątków.** Wątki puli czekają uśpione na zmiennej warunkowej, a ich budzenie i tworzenie nowych wątków kosztuje około 10 μs na wątek (punkt 7.7.5). Krótkie aktywne oczekiwanie przed uśpieniem, stosowane np. w bibliotece OpenMP kompilatora GCC, skróciłoby czas dla małych obrazów, dla których W4 osiąga tylko 2,58×.

7. **Nakładanie przesyłu i obliczeń na GPU.** Podział obrazu na pasy i przesyłanie kolejnego pasa w czasie obliczeń na poprzednim (kilka kolejek OpenCL) ukryłoby część z 0,713 ms + 0,630 ms przesyłu dla 4K. Dodatkowo pamięć przypięta (*pinned*) przyspiesza sam przesył przez PCIe, a przy przetwarzaniu serii obrazów wyniki pośrednie mogłyby pozostawać na karcie.

8. **Wektorowe skalowanie we wszystkich wersjach.** Skalowanie skalarne trwało w W1 i W2 około 3,17 ms dla 4K, czyli około 20 % czasu W2. Wersja wektorowa z W3 trwała 1,21 ms; przeniesienie jej do W2 skróciłoby W2 o około 2 ms.

7.12 Podsumowanie i wnioski

Zaimplementowano program wykrywający krawędzie w pięciu wersjach: sekwencyjnej W1 i czterech wersjach, z których każda dodaje do W1 jeden rodzaj przyspieszenia. Wszystkie wersje dają wynik identyczny co do bitu z W1. Na podstawie pomiarów na procesorze AMD Ryzen 9 9900X i karcie GTX 1050 Ti sformułowano następujące wnioski.

1. Każdy rodzaj przyspieszenia skrócił sam algorytm wielokrotnie: dla obrazu 4K i maski 3×3 z 76,2 ms do 16,0 ms (W2), 8,40 ms (W3), 8,49 ms (W4) i 6,73 ms (W5). Żadna wersja nie osiągnęła granicy teoretycznej swojego mechanizmu (8 dla AVX2, 24 dla wątków).

2. Zmiana algorytmu na jednym rdzeniu (W2) dała dla maski 7×7 przyspieszenie 13,7× – więcej niż wektoryzacja (12,6×) i 24 wątki (11,6×). Wybór algorytmu o niższej złożoności jest więc pierwszym krokiem przed zrównolegleniem.

3. Wersja wielowątkowa W4 osiągnęła na 24 wątkach przyspieszenie 9,05 dla 4K i tylko 5,34 dla 8K. Udział części sekwencyjnej, głównie jednowątkowego przydziału buforów, wyniósł według prawa Amdahla 5,6 % i 13,3 %. Wątki SMT powyżej 12 dały dla 4K i 8K mniej niż 10 % dodatkowego zysku.

4. Spośród mechanizmów zarządzania wątkami największy wpływ miały: sposób synchronizacji (blokada przy każdym pikselu 39,4× wolniej, fałszywe współdzielenie 3,79× wolniej niż lokalne maksimum), rozmiar fragmentu (fragment 256 wierszy 15,1 ms wobec 7,38 ms dla 1 wiersza) i przypisanie wątków do rdzeni (do 1,5× wolniej przy złym przypisaniu). Pula wątków, bariera między etapami i atomowa zmienna wspólna miały wpływ mniejszy niż rozrzut pomiaru dla obrazów 4K.

5. Karta graficzna (W5) była najszybsza dla obrazów od 2560×1440 wzwyż i przy dużych maskach (26,9× dla 7×7 i 4K). Dla mniejszych obrazów szybsza była wersja wektorowa W3. Przesył przez PCIe zajmował około 20 % czasu W5. Mniejsza liczba wątków GPU, z których każdy liczy kilka pikseli, skróciła obliczenia na karcie 2,4×.

6. Dla obrazów większych niż pamięć L3 (powyżej około 5 Mpix) wersje jednowątkowe zwalniają wielokrotnie: W3 potrzebowała 7,42 ms dla 4K i 31,7 ms dla obrazu 1,8 raza większego.

7. Cały program przyspieszył najwyżej 1,38× dla 4K, ponieważ wczytanie i zapis JPG stanowią 68,8 % czasu W1 i ograniczają przyspieszenie do 1,45 (prawo Amdahla). Dalsza praca nad samym algorytmem ma sens tylko po przyspieszeniu zapisu wyniku.

8. Większa maska nie daje lepszego wyniku na obrazie bez szumu (Roberts $F = 0,995$, Sobel 7×7 $F = 0,932$), ale przy silnym szumie ($\sigma = 40$) Sobel 7×7 ($F = 0,931$) jest znacznie lepszy od 3×3 ($F = 0,675$).

Ograniczenia: wyniki dotyczą jednego komputera z dwoma blokami rdzeni i jednej karty graficznej klasy konsumenckiej, obrazów w skali szarości do 59 Mpix i masek gradientowych do 7×7. Na procesorze z jednym blokiem rdzeni lub nowszej karcie graficznej proporcje między wersjami mogą być inne.

Literatura

[1] Gonzalez R. C., Woods R. E., *Digital Image Processing*, 4th ed., Pearson, 2018.

[2] Tadeusiewicz R., Korohoda P., *Komputerowa analiza i przetwarzanie obrazów*, Wydawnictwo Fundacji Postępu Telekomunikacji, Kraków, 1997.

[3] Czech Z., *Wprowadzenie do obliczeń równoległych*, PWN, Warszawa, 2010.

[4] Williams A., *C++ Concurrency in Action*, 2nd ed., Manning, 2019.

[5] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021.

[6] Amdahl G. M., *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS Conference Proceedings, vol. 30, 1967.

[7] Drepper U., *What Every Programmer Should Know About Memory*, Red Hat, 2007.

[8] Intel Intrinsics Guide, `https://www.intel.com/content/www/us/en/docs/intrinsics-guide/`, 09.11.2026.

[9] Khronos Group, *The OpenCL Specification*, Version 1.2, 2012.

[10] Abdou I. E., Pratt W. K., *Quantitative design and evaluation of enhancement/thresholding edge detectors*, Proceedings of the IEEE, vol. 67, no. 5, 1979, s. 753–763.

[11] Hoefler T., Belli R., *Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses when Reporting Performance Results*, Proceedings of SC'15, ACM, 2015.

[12] `https://en.cppreference.com/w/cpp/thread`, 19.10.2026.

[13] Qt 6 Documentation, `https://doc.qt.io/qt-6/`, 09.11.2026.

[14] `https://github.com/nothings/stb`, 19.10.2026.

[15] NASA Earth Observatory, *Blue Marble: Next Generation*, `https://science.nasa.gov/earth/earth-observatory/blue-marble-next-generation/`, 08.11.2026.

[16] Kodak Lossless True Color Image Suite, `https://r0k.us/graphics/kodak/`, 08.11.2026.